

Final Project – Machine Learning

Prediction of Student Academic Performance

Nicolás Carrión

1. Introduction

The PISA (*Programme for International Student Assessment*) assessment, developed by the Organisation for Economic Co-operation and Development (OECD), is one of the most important international studies for measuring the academic performance of 15-year-old students in key areas such as mathematics, reading, and science. PISA aims to evaluate the extent to which students have acquired the knowledge and skills necessary to participate fully in modern society, focusing not only on the acquisition of knowledge but also on the ability to apply it in real-world situations [1].

In addition to assessing academic performance, PISA collects a large amount of contextual information related to students' socioeconomic, family, school, and personal factors. This information makes it possible to study the influence of various factors on academic achievement, contributing to the identification of patterns that may be useful for designing strategies and policies aimed at improving educational quality.

However, the large number of variables available in the PISA questionnaires presents an analytical challenge due to high dimensionality, the presence of missing values, and potential redundancy among variables. In this context, machine learning techniques provide tools capable of modeling complex relationships and generating accurate predictions from large volumes of data.

In this project, the PISA 2022 dataset will be used with the objective of predicting students' mathematics performance, represented by the `PV_MATH_MEAN` variable, which corresponds to the average of the ten plausible mathematics values. To achieve this, a feature selection process will first be conducted in order to identify the most relevant variables and reduce the dimensionality of the problem. Subsequently, an Exploratory Data Analysis (EDA) will be performed on the selected variables, and two machine learning models will be trained: a Random Forest Regressor and an Artificial Neural Network.

Finally, the performance of both models will be evaluated using regression metrics and compared through statistical tests to determine whether significant differences exist between their predictive capabilities. This work constitutes an initial approach to the study of academic performance through machine learning techniques and may serve as a foundation for future, more extensive research, including the development of thesis projects focused on interpreting factors associated with student achievement.

2. Data Ingestion

First, we extract the data from the ZIP file available at https://webfs.oecd.org/pisa2022/SCH_QQQ_SAS.zip. To improve processing efficiency, the dataset is stored in Parquet format. At the same time, we split the data into training and test sets.

```
In [1]: import pandas as pd

def sas_to_parquet():
    df = pd.read_sas(
        "data/CY08MSP_STU_QQQ.SAS7BDAT",
        format="sas7bdat"
    )
    print("Loaded SAS file into DataFrame")

    for col in df.select_dtypes(include=[object]).columns:
        df[col] = df[col].apply(lambda x: x.decode("utf-8") if isinstance(x, bytes) else x)

    df.to_parquet("data/pisa2022.parquet", index=False)
    print("Converted SAS file to Parquet format and saved as data/pisa2022.parquet")
```

```
In [2]: sas_to_parquet()
```

```
Loaded SAS file into DataFrame
Converted SAS file to Parquet format and saved as data/pisa2022.parquet
```

```
In [1]: import pyarrow.dataset as ds
from sklearn.model_selection import train_test_split
```

```
def split_data(test_size=0.2, random_state=42):
    dataset = ds.dataset('data/pisa2022.parquet', format='parquet')
    table = dataset.to_table()

    train_table, test_table = train_test_split(
        table.to_pandas(),
        test_size=test_size,
        random_state=random_state
    )

    print(f"Train rows: {len(train_table)} ({(1-test_size)*100:.1f}%)")
    print(f"Test rows: {len(test_table)} ({test_size*100:.1f}%)")

    return train_table, test_table

def save_split(train_df, test_df):
    train_df.to_parquet('data/train.parquet', index=False)
    print("Saved train split to data/train.parquet")

    test_df.to_parquet('data/test.parquet', index=False)
    print("Saved test split to data/test.parquet")
```

In [2]: `save_split(*split_data())`

```
Train rows: 490995 (80.0%)
Test rows: 122749 (20.0%)
Saved train split to data/train.parquet
Saved test split to data/test.parquet
```

In [4]: `import pyarrow.parquet as pq`

```
table = pq.read_table("data/pisa2022.parquet")

print("Shape:", table.num_rows, "x", table.num_columns)
print("Estimated memory usage (MB):", table.nbytes / 1024**2)
```

```
Shape: 613744 x 1278
Estimated memory usage (MB): 6100.628432273865
```

Given the size of the dataset, loading all records into memory is not practical. Consequently, the analysis must be conducted either on selected subsets of columns or on a reduced sample of rows (for example, 5% of the data, corresponding to approximately 30k records).

In [5]: `print(f"5% of the data: {table.num_rows * 0.05} rows")`

```
5% of the data: 30687.2 rows
```

3. ETL

Next, we clean the dataset to retain only the features and the target variable, `PV_MATH_MEAN`. Since this variable is not directly available in the original dataset, it must first be computed and added. To reduce redundancy, all other plausible values (including those for mathematics, reading, and science) are removed, keeping only `PV_MATH_MEAN` as the target variable.

In [10]: `import pyarrow.parquet as pq`
`import pyarrow.compute as pc`
`import pyarrow as pa`

```
def extract_data():
    train = pq.read_table('data/train.parquet')
    test = pq.read_table('data/test.parquet')
    return {'train': train, 'test': test}

def transform_data(table):
    columns = table.column_names

    math_cols = [f'PV{i}MATH' for i in range(1, 11)]
    available = [c for c in math_cols if c in columns]

    if available:
        arrays = [table[c] for c in available]
        mean_array = arrays[0]
        for arr in arrays[1:]:
            mean_array = pc.add(mean_array, arr)
        mean_array = pc.divide(mean_array, len(available))
        table = table.append_column("PV_MATH_MEAN", mean_array)
        print(f"Transformed {len(available)} PV_MATH columns into PV_MATH_MEAN.")

    pv_cols = [c for c in columns if c.startswith("PV") and c != "PV_MATH_MEAN"]
```

```

print(f"Dropping {len(pv_cols)} PV columns")
table = table.drop_columns(pv_cols)

return table

def load_data(table, file_name):
    pq.write_table(table, file_name)
    print(f"Saved transformed data to {file_name}")

```

```

In [11]: for name, table in extract_data().items():
        table = transform_data(table)
        load_data(table, file_name=f"data/{name}_transformed.parquet")

```

Transformed 10 PV_MATH columns into PV_MATH_MEAN.
Dropping 110 PV columns
Saved transformed data to data/train_transformed.parquet
Transformed 10 PV_MATH columns into PV_MATH_MEAN.
Dropping 110 PV columns
Saved transformed data to data/test_transformed.parquet

4. Feature Selection

The original dataset contains more than 1,200 variables; therefore, dimensionality reduction is required before model training. The first step is to encode categorical predictors using either one-hot encoding or frequency encoding, depending on their cardinality and analytical context.

The PISA 2022 codebook is available at https://webfs.oecd.org/pisa2022/CY08MSP_CODEBOOK_27thJune24.xlsx. This codebook was used to construct a CSV file containing each variable and the number of entries associated with it. Since additional entries typically represent response categories, variables with more than one entry are treated as candidate categorical variables.

```

In [1]: import pandas as pd
import pyarrow.parquet as pq

var_counts = pd.read_csv('dataset_var_counts.csv', sep=';')
parquet_file = pq.ParquetFile('data/train_transformed.parquet')
df_cols = parquet_file.schema.names

# drop columns that are not in the dataframe
var_counts = var_counts[var_counts['VAR'].isin(df_cols)]

print(var_counts.shape)
display(var_counts)

```

(1168, 2)

	VAR	COUNT
0	CNTSCHID	1
1	CNTSTUID	1
2	CYC	1
3	AGE	1
4	W_FSTUWT	1
...	...	...
1273	OCOD1	625
1274	OCOD2	625
1275	OCOD3	625
1276	PROGN	664
1277	STRATUM	1318

1168 rows × 2 columns

```

In [2]: numerical_columns = var_counts[var_counts['COUNT'] == 1]['VAR'].tolist()
len(numerical_columns)

```

Out[2]: 88

```

In [3]: categorical_columns = var_counts[var_counts['COUNT'] > 1]['VAR'].tolist()

```

```
var_counts[var_counts['COUNT'] > 1]
```

Out[3]:

	VAR	COUNT
198	OECD	3
199	ADMINMODE	3
200	Option_CT	3
201	Option_FL	3
202	Option_ICTQ	3
...	...	...
1273	OCOD1	625
1274	OCOD2	625
1275	OCOD3	625
1276	PROGN	664
1277	STRATUM	1318

1080 rows × 2 columns

Although the number of categories can be inferred from the codebook, it is necessary to verify this information directly in the training dataset.

```
In [5]: import pandas as pd
import pyarrow.parquet as pq
from collections import defaultdict

var_counts = pd.read_csv('dataset_var_counts.csv', sep=';')

parquet_file = pq.ParquetFile('data/train_transformed.parquet')

df_cols = parquet_file.schema.names

var_counts = var_counts[var_counts["VAR"].isin(df_cols)].copy()

numerical_columns = var_counts.loc[
    var_counts["COUNT"] == 1, "VAR"
].tolist()

categorical_columns = var_counts.loc[
    var_counts["COUNT"] > 1, "VAR"
].tolist()

schema = parquet_file.schema_arrow

char_columns = [
    field.name
    for field in schema
    if str(field.type) in ("string", "large_string")
]

unique_sets = defaultdict(set)
null_counts = defaultdict(int)

batch_size = 100_000

for batch in parquet_file.iter_batches(batch_size=batch_size):
    for col_name, arr in zip(batch.schema.names, batch.columns):
        null_counts[col_name] += arr.null_count

    df_batch = batch.to_pandas(
        split_blocks=True,
        self_destruct=True
    )

    for col in df_batch.columns:
        values = df_batch[col].dropna().unique()

        if len(values):
            unique_sets[col].update(values)

unique_counts = pd.Series(
    {col: len(vals) for col, vals in unique_sets.items()})
```

```

)

null_counts = pd.Series(dict(null_counts))

final_df = pd.DataFrame({
    "variable": var_counts["VAR"].values,
    "count_csv": var_counts["COUNT"].values
})

final_df["unique_count_parquet"] = (
    final_df["variable"].map(unique_counts)
)

final_df["null_count_parquet"] = (
    final_df["variable"].map(null_counts).fillna(0).astype("int64")
)

final_df["is_categorical_csv"] = (
    final_df["variable"].isin(categorical_columns)
)

final_df["is_numerical_csv"] = (
    final_df["variable"].isin(numerical_columns)
)

final_df["is_char"] = (
    final_df["variable"].isin(char_columns)
)

def infer_type(row):
    if row["is_char"]:
        return "char"
    elif row["is_categorical_csv"]:
        return "categorical"
    else:
        return "numerical"

final_df["final_type"] = final_df.apply(
    infer_type,
    axis=1
)

final_df.drop(columns=['is_categorical_csv', 'is_numerical_csv', 'is_char'], inplace=True)

final_df

```

Out[5]:

	variable	count_csv	unique_count_parquet	null_count_parquet	final_type
0	CNTSCHID	1	21567	0	numerical
1	CNTSTUID	1	490995	0	numerical
2	CYC	1	1	0	char
3	AGE	1	16	0	numerical
4	W_FSTUWT	1	45875	0	numerical
...	...	...	...	...	...
1163	OCOD1	625	624	0	char
1164	OCOD2	625	624	0	char
1165	OCOD3	625	607	0	char
1166	PROGN	664	438	0	char
1167	STRATUM	1318	1014	0	char

1168 rows × 5 columns

Using this summary table, variables with no useful variation are identified and removed. Specifically, columns with only one unique value are discarded. Character variables and variables identified as categorical in the codebook are treated as categorical predictors. When a variable is marked as categorical in the codebook but has more unique values in the Parquet than expected, it is treated as numerical, since this pattern suggests that the variable may represent a continuous measure with special missing-value codes.

For simplicity, variables containing missing values are also excluded from the analysis. Properly handling missing data would require a more detailed assessment of the underlying missingness mechanisms and the selection of appropriate imputation or

treatment strategies, which is beyond the scope of this work.

```
In [11]: # Columns to drop for having only one unique value in the parquet file (meaning they are constant) or having null
columns_to_drop = set()
mask1 = final_df['unique_count_parquet'] == 1
mask1 = mask1 | (final_df['null_count_parquet'] > 0)

display(final_df[mask1])

columns_to_drop.update(final_df[mask1]['variable'].tolist())
print(f"Columns to drop: {columns_to_drop}")

# Columns that are character type meaning that should be treated as categorical
char_columns = set()
mask2 = (final_df['final_type'] == 'char') & (~mask1)

display(final_df[mask2])

char_columns.update(final_df[mask2]['variable'].tolist())
print(f"Character columns: {char_columns}")

# Columns that are categorical but have more unique values in the parquet file than in the csv file, meaning that
cat_exceptions = set()
mask3 = (final_df['final_type'] == 'categorical') & (~mask1) & (~mask2) & (final_df['unique_count_parquet'] > fi

display(final_df[mask3])

cat_exceptions.update(final_df[mask3]['variable'].tolist())
print(f"Categorical exceptions: {cat_exceptions}")

# Columns that are categorical without exceptions, meaning that they should be treated as categorical
categorical_columns = set()
mask4 = (final_df['final_type'] == 'categorical') & (~mask1) & (~mask2) & (~mask3)

display(final_df[mask4])

categorical_columns.update(final_df[mask4]['variable'].tolist())
print(f"Categorical columns: {categorical_columns}")
```

	variable	count_csv	unique_count_parquet	null_count_parquet	final_type
2	CYC	1	1	0	char
98	ST326Q07JA	6	30	478064	categorical
99	ST326Q08JA	6	27	478379	categorical
100	ST326Q09JA	6	29	478210	categorical
101	ST326Q10JA	6	28	479040	categorical
...	...	...	...	...	...
1142	ST021Q01TA	23	17	449590	categorical
1143	FCFMLRTY	23	17	354603	categorical
1144	ISCEDP	62	7	3	categorical
1146	LANGTEST_QQQ	73	55	8699	categorical
1147	LANGTEST_PAQ	73	16	416462	categorical

1049 rows × 5 columns

Columns to drop: {'FL172Q03JA', 'ST324Q04JA', 'WB155Q03HA', 'ST276Q04JA', 'PA189Q09JA', 'PA003Q15JA', 'ST276Q02JA', 'ST285Q02JA', 'FL167Q04HA', 'ST353Q05JA', 'ST305Q06JA', 'ST324Q07JA', 'ST300Q05JA', 'ST345Q05JA', 'ST343Q01JA', 'ST326Q09JA', 'ST034Q04TA', 'ST345Q07JA', 'ST349Q01JA', 'IC177Q01JA', 'PA003Q03TA', 'EMPATAGR', 'ST293Q09JA', 'ST006Q03JA', 'EXP021ST', 'ST348Q05JA', 'ST265Q02JA', 'ST256Q10JA', 'ST268Q08JA', 'PA185Q01JA', 'FLMULTSB', 'LEARRES', 'ST275Q08WA', 'WB162Q07HA', 'IC170Q07JA', 'IC172Q01JA', 'FL167Q02HA', 'IC176Q07JA', 'ST305Q08JA', 'PA188Q01JA', 'ST300Q02JA', 'ST313Q07JA', 'FL174Q06JA', 'WB155Q04HA', 'ST354Q07JA', 'FAMSUPSL', 'ST334Q06JA', 'PA041Q01TA', 'ST034Q05TA', 'ST283Q05JA', 'WB154Q02HA', 'SOCCON', 'ST327Q01JA', 'PA007Q02TA', 'WB154Q06HA', 'ST292Q04JA', 'ST327Q03JA', 'ICTHOME', 'ST337Q02JA', 'FL164Q15HA', 'ST265Q01JA', 'FL163Q05HA', 'FISCED', 'ST276Q03JA', 'ST272Q01JA', 'WB162Q06HA', 'WB171Q01HA', 'ST340Q10JA', 'ST273Q07JA', 'MATHEFF', 'ST335Q01JA', 'WB165Q01HA', 'FL160Q02HA', 'IC183Q04JA', 'ST354Q08JA', 'CREATOOS', 'WB154Q09HA', 'PA167Q04HA', 'WB168Q03HA', 'ST340Q07JA', 'PA189Q02JA', 'ST292Q01JA', 'ST034Q06TA', 'IC170Q04JA', 'ST322Q02JA', 'IC179Q05JA', 'IC175Q02JA', 'IC174Q10JA', 'IC183Q05JA', 'ST276Q05JA', 'ST273Q02JA', 'ST268Q03JA', 'IC172Q07JA', 'ST267Q04JA', 'IC171Q05JA', 'PA183Q03JA', 'WB151Q01HA', 'PA196Q02WA', 'ST315Q05JA', 'ST290Q02WA', 'WB153Q01HA', 'WB158Q01HA', 'ST307Q04JA', 'FCFMLRTY', 'ST311Q09JA', 'ST276Q08JA', 'ST334Q09JA', 'ST285Q04JA', 'WB168Q01HA', 'ST324Q14JA', 'ST301Q10JA', 'CREATSCH', 'FL159Q03HA', 'ST303Q03JA', 'FL161Q02HA', 'FL170Q02JA', 'ST005Q01JA', 'ST254Q05JA', 'PQMIMP', 'ST330Q08WA', 'ST307Q09JA', 'ICTREG', 'ST315Q07JA', 'PA189Q10JA', 'ST289Q08WA', 'IC171Q04JA', 'ISCEDP', 'PA001Q03TA', 'PA009Q01NA', 'ST330Q07WA', 'ST062Q01TA', 'ST268Q07JA', 'FL174Q02JA', 'IC180Q01JA', 'IC180Q07JA', 'WB177Q04HA', 'PA196Q03WA', 'ST267Q08JA', 'ST293Q08JA', 'IC178Q04JA', 'ST265Q03JA', 'IC172Q05JA', 'PA189Q04JA', 'ST303Q06JA', 'IC174Q01JA', 'ST251Q07JA', 'ST267Q07JA', 'FL163Q03HA', 'WB173Q01HA', 'ST334Q02JA', 'PA008Q06NA', 'ST261Q07JA', 'IC175Q03JA', 'ST297Q07JA', 'ST330Q09WA', 'ST127Q03TA', 'FEELLAH', 'ST313Q10JA', 'ST275Q01WA', 'ST307Q02JA', 'ST283Q02JA', 'WB153Q03HA', 'ICTDISTR', 'WB178Q05HA', 'PA009Q05NA', 'ST351Q02JA', 'ST348Q04JA', 'ST007Q01JA', 'ST300Q09JA', 'PA186Q08JA', 'ST334Q05JA', 'PQSCHOOL', 'ST251Q04JA', 'ST355Q05JA', 'ST340Q02JA', 'ST008Q01JA', 'WB173Q03HA', 'ST293Q05JA', 'ST261Q02JA', 'ST353Q02JA', 'ST315Q01JA', 'ST295Q01JA', 'FL164Q08HA', 'ST256Q06JA', 'WB178Q07HA', 'IC176Q04JA', 'ST311Q06JA', 'ST290Q08WA', 'ST305Q02JA', 'WB178Q04HA', 'IC179Q06JA', 'IC181Q03JA', 'FL172Q01JA', 'ST326Q10JA', 'IC172Q04JA', 'PA197Q05WA', 'WB155Q06HA', 'ST305Q05JA', 'ST285Q01JA', 'EMOCOAGR', 'ST354Q06JA', 'ST261Q10JA', 'ST297Q05JA', 'ST352Q08JA', 'IC183Q01JA', 'ST345Q06JA', 'IC170Q01JA', 'ST295Q05JA', 'ST263Q04JA', 'ST337Q04JA', 'ST261Q03JA', 'PA009Q07NA', 'ST283Q04JA', 'ST356Q01JA', 'ST303Q04JA', 'PA175Q01HA', 'ST338Q02JA', 'ST291Q03JA', 'ST270Q01JA', 'WB155Q07HA', 'IC176Q02JA', 'ST290Q04WA', 'ST305Q01JA', 'BELONG', 'ICTWKDY', 'ST255Q01JA', 'ST334Q08JA', 'ST283Q07JA', 'PA186Q02JA', 'PA009Q11NA', 'IC181Q01JA', 'WB162Q04HA', 'ST309Q09JA', 'ST326Q06JA', 'ST256Q01JA', 'WB154Q04HA', 'ST296Q03JA', 'ST301Q04JA', 'ST307Q05JA', 'IC174Q03JA', 'ST301Q08JA', 'PA018Q02NA', 'FL164Q09HA', 'PA009Q02NA', 'ST268Q01JA', 'ST338Q03JA', 'ST307Q10JA', 'ST059Q02JA', 'ST273Q05JA', 'FL164Q10HA', 'IC179Q01JA', 'FL164Q06HA', 'PA175Q02HA', 'ST038Q09JA', 'CREATOPN', 'WORKPAY', 'PA006Q05TA', 'ST307Q03JA', 'SKIPPING', 'ST275Q02WA', 'LANGTEST_PAQ', 'ST337Q01JA', 'FL167Q03HA', 'PA006Q12HA', 'PA001Q01TA', 'ST267Q02JA', 'PA003Q05IA', 'ST254Q02JA', 'IC171Q06JA', 'ST294Q03JA', 'PA007Q09NA', 'PA186Q05JA', 'FL171Q05JA', 'ST341Q04JA', 'ST345Q01JA', 'ST338Q06JA', 'ST275Q04WA', 'WB167Q01HA', 'PA188Q06JA', 'IC184Q04JA', 'CREATOR', 'PA006Q08TA', 'ST353Q03JA', 'IC173Q04JA', 'WB155Q10HA', 'WB163Q03HA', 'ST059Q01TA', 'WB156Q01HA', 'ST331Q01JA', 'ST348Q02JA', 'PA006Q01TA', 'PA175Q04JA', 'ICTEFFIC', 'ST340Q01JA', 'FLFAMILY', 'WB162Q02HA', 'ST338Q07JA', 'COGACMCO', 'FL171Q02JA', 'ST297Q06JA', 'FL159Q02HA', 'ST341Q05JA', 'ST342Q03JA', 'ST289Q05WA', 'PA009Q09NA', 'PA006Q02TA', 'IC176Q01JA', 'IC174Q07JA', 'IC171Q01JA', 'ST230Q01JA', 'FL164Q01HA', 'PA183Q07JA', 'FL171Q04JA', 'WB178Q03HA', 'PA006Q09TA', 'IC182Q03JA', 'ST340Q05JA', 'PA188Q03JA', 'ST341Q01JA', 'ANXMAT', 'ST291Q08JA', 'ST334Q03JA', 'ST313Q05JA', 'RELATST', 'PA009Q08NA', 'IC174Q08JA', 'IC182Q02JA', 'ST293Q03JA', 'PA003Q18WA', 'ST294Q02JA', 'PA185Q04JA', 'ST019CQ01T', 'ST343Q09JA', 'ST290Q05WA', 'PA186Q06JA', 'EXPECEDU', 'ST283Q09JA', 'PA008Q10NA', 'ST297Q01JA', 'ST038Q03NA', 'IC172Q02JA', 'ST256Q03JA', 'PA194Q01JA', 'ST301Q05JA', 'ST348Q07JA', 'PA183Q01JA', 'ST354Q02JA', 'ST331Q02JA', 'ST343Q04JA', 'ST289Q01HA', 'EXP0FA', 'ST311Q07JA', 'ST291Q02JA', 'ST339Q06JA', 'ST266Q04JA', 'FL150Q03TA', 'ST343Q02JA', 'ST008Q04JA', 'PA008Q09NA', 'PA006Q06TA', 'PA007Q05TA', 'PA007Q07TA', 'WB150Q01HA', 'ST350Q01JA', 'FL163Q02HA', 'ST251Q06JA', 'ST343Q06JA', 'IC170Q06JA', 'WB155Q08HA', 'WB163Q05HA', 'ST355Q01JA', 'ST343Q05JA', 'WB166Q01HA', 'FLSCHOOL', 'ST340Q08JA', 'PA007Q13NA', 'PA185Q03JA', 'ST283Q06JA', 'ST251Q02JA', 'FL164Q04HA', 'WB171Q04HA', 'PA177Q03HA', 'ST340Q04JA', 'BULLIED', 'ST290Q06NA', 'PA189Q05JA', 'STUDYHMM', 'MISCED', 'ST008Q02JA', 'CREATFAM', 'ST290Q03WA', 'ICTAVSCH', 'ST276Q09JA', 'ST297Q03JA', 'PA177Q05HA', 'ST326Q01JA', 'ST311Q08JA', 'WB154Q03HA', 'ST355Q07JA', 'ST313Q08JA', 'ACCESSFP', 'ST273Q04JA', 'ST261Q09JA', 'ST338Q01JA', 'SOCONPA', 'ST335Q03JA', 'PA189Q01JA', 'IC172Q06JA', 'ST351Q03JA', 'ST354Q10JA', 'FL167Q05HA', 'IC180Q08JA', 'IC173Q01JA', 'ST300Q04JA', 'ST348Q06JA', 'PAREXPT', 'ST276Q10JA', 'ST342Q04JA', 'ST309Q10JA', 'ST289Q02JA', 'ST293Q06JA', 'PA188Q09JA', 'FL166Q02HA', 'ST351Q08JA', 'ST309Q07JA', 'PA185Q09JA', 'ST308Q06NA', 'ST263Q08JA', 'IC184Q02JA', 'ST324Q02JA', 'ST336Q03JA', 'ST342Q02JA', 'PAREDINT', 'ST326Q12JA', 'ST250Q03JA', 'FAMSUP', 'IC176Q06JA', 'ST292Q02JA', 'ST336Q07JA', 'ST034Q03TA', 'ICTSUBJ', 'PA196Q04WA', 'FL163Q01HA', 'ST275Q07WA', 'ST335Q06JA', 'IC179Q04JA', 'IC183Q12JA', 'ST352Q02JA', 'ST326Q08JA', 'PQMCA', 'ST315Q04JA', 'ICTWKEND', 'WB166Q03HA', 'ST267Q05JA', 'ST341Q02JA', 'PARINVOL', 'PA186Q04JA', 'PA183Q05JA', 'WB168Q04HA', 'ST258Q01JA', 'ST303Q02JA', 'FL166Q07HA', 'WB162Q03HA', 'ST309Q02JA', 'PA177Q07HA', 'ST327Q04JA', 'WB163Q07HA', 'PA189Q03JA', 'PA186Q07JA', 'ST261Q06JA', 'ST352Q07JA', 'PA007Q14NA', 'IC176Q08JA', 'SISCO', 'ST290Q07WA', 'BMMJ1', 'ST340Q06JA', 'ST326Q05JA', 'ST289Q04JA', 'PA008Q05TA', 'PA185Q02JA', 'COGACRCO', 'ST254Q01JA', 'IC179Q02JA', 'PA183Q04JA', 'WB162Q01HA', 'FL171Q09JA', 'PA188Q08JA', 'ST336Q04JA', 'PA001Q02TA', 'ESCS', 'FL172Q06JA', 'ST289Q03WA', 'IC183Q03JA', 'WB162Q09HA', 'ST295Q02JA', 'IC183Q02JA', 'PA003Q11JA', 'ST285Q05JA', 'ST347Q02JA', 'IC180Q03JA', 'ST038Q04NA', 'ST338Q05JA', 'IC170Q05JA', 'CREATEFF', 'ST354Q01JA', 'ST352Q05JA', 'FLCONICT', 'ST313Q01JA', 'ST270Q02JA', 'ST270Q03JA', 'PA183Q06JA', 'IC174Q04JA', 'ST311Q03JA', 'ST341Q03JA', 'ST330Q06WA', 'FL164Q11HA', 'ST016Q01NA', 'ST326Q04JA', 'ST322Q01JA', 'ST303Q08JA', 'ST354Q03JA', 'IC180Q04JA', 'ST301Q07JA', 'ST343Q08JA', 'ST330Q05WA', 'ST276Q07JA', 'PA185Q10JA', 'WB166Q02HA', 'FL171Q08JA', 'ST354Q04JA', 'ST351Q07JA', 'PA005Q01TA', 'PA188Q10JA', 'ST289Q11WA', 'WB168Q02HA', 'WB162Q08HA', 'IC173Q03JA', 'IC171Q03JA', 'FL169Q04HA', 'ICTQUAL', 'ST338Q08JA', 'ST291Q07JA', 'FL170Q06JA', 'FL174Q04JA', 'ST351Q05JA', 'ST352Q06JA', 'IMAGINE', 'FL166Q05HA', 'ST250Q05JA', 'ICTAVHOM', 'HISCED', 'ST315Q03JA', 'WB177Q01HA', 'FL174Q07JA', 'ST019BQ01T', 'WB155Q09HA', 'ST062Q03TA', 'PA009Q04NA', 'PA195Q01JA', 'LANGTEST_QQQ', 'ST261Q11JA', 'PA003Q02TA', 'ST343Q03JA', 'ST003D02T', 'ST291Q05JA', 'ST330Q11WA', 'ST290Q01WA', 'ST355Q02JA', 'ST275Q09WA', 'FL171Q10JA', 'ST347Q01JA', 'ST038Q05NA', 'ST259Q02JA', 'PA006Q11TA', 'IC177Q07JA', 'SCHRISK', 'PROBSELF', 'ST305Q03JA', 'ST294Q01JA', 'ST300Q06JA', 'ST293Q04JA', 'ST292Q05JA', 'ST296Q02JA', 'PA185Q07JA', 'FL162Q04HA', 'ST125Q01NA', 'FL164Q07HA', 'ST270Q04JA', 'WB160Q01HA', 'PA167Q03HA', 'ST293Q01JA', 'PA177Q04HA', 'PA186Q01JA', 'ST034Q01TA', 'FL166Q01HA', 'ST292Q03JA', 'WB164Q01HA', 'ST342Q08JA', 'ST019AQ01T', 'PA197Q02WA', 'PA182Q01HA', 'ST340Q09JA', 'ST352Q01JA', 'WB162Q05HA', 'ST275Q06WA', 'IC178Q07JA', 'ST296Q01JA', 'ST266Q03JA', 'PA167Q05HA', 'FL170Q04JA', 'ST127Q01TA', 'ST266Q01JA', 'PA180Q01HA', 'PA197Q04WA', 'ST324Q05JA', 'IC184Q03JA', 'PA183Q02JA', 'PA006Q03TA', 'CREATOP', 'ST289Q07JA', 'IC178Q06JA', 'PA008Q01TA', 'IC179Q03JA', 'FL164Q05HA', 'PA197Q01HA', 'WB176Q01HA', 'EFFORT1', 'IC180Q02JA', 'FL161Q01HA', 'IC178Q03JA', 'ATTIMPP', 'PA006Q10TA', 'MATHPERS', 'PA008Q04TA', 'ST334Q10JA', 'ST343Q07JA', 'FL167Q06JA', 'ST338Q04JA', 'IC178Q02JA', 'ST354Q09JA', 'ST327Q02JA', 'ST343Q10JA', 'PERSEVAGR', 'ST353Q07JA', 'ST345Q09JA', 'ST126Q01TA', 'ST300Q07JA', 'ST292Q06JA', 'ST353Q04JA', 'ST283Q03JA', 'PA007Q11NA', 'ST295Q04JA', 'ST006Q02JA', 'IC182Q01JA

A', 'FL170Q03JA', 'ST322Q03JA', 'ST313Q06JA', 'WB171Q02HA', 'ST250Q01JA', 'IC181Q02JA', 'ST335Q05JA', 'ST289Q06JA', 'ST273Q01JA', 'ST348Q08JA', 'ST276Q01JA', 'ST315Q06JA', 'ST355Q06JA', 'FL169Q08JA', 'WB177Q02HA', 'ST260Q03JA', 'ST038Q10JA', 'ST313Q09JA', 'PA018Q03NA', 'WB154Q01HA', 'ST327Q05JA', 'ST336Q06JA', 'ST351Q04JA', 'IC177Q06JA', 'EXPWB', 'FL169Q05JA', 'ST303Q01JA', 'FL171Q12JA', 'IC176Q05JA', 'PA006Q07TA', 'ST339Q04JA', 'MATHEASE', 'FL166Q03HA', 'FL169Q06JA', 'ST254Q06JA', 'ST256Q08JA', 'ST334Q01JA', 'IC183Q14JA', 'ST289Q14JA', 'IC180Q06JA', 'ST265Q04JA', 'FL162Q02HA', 'FL172Q05JA', 'IC183Q09JA', 'ST315Q02JA', 'SDLEFF', 'ST250Q04JA', 'ST353Q01JA', 'ST062Q02TA', 'ST353Q08JA', 'PA007Q06TA', 'ST342Q06JA', 'ST273Q03JA', 'FL174Q03JA', 'ST266Q02JA', 'PA189Q06JA', 'PA196Q01WA', 'STUBMI', 'ST006Q05JA', 'DISCLIM', 'ST295Q03JA', 'HOMEPOS', 'FL162Q03HA', 'ST307Q08JA', 'PA177Q02HA', 'ST324Q13JA', 'FL171Q01JA', 'WORKHOME', 'PA197Q03WA', 'PA177Q08HA', 'IC183Q07JA', 'SCHSUST', 'WB155Q02HA', 'WB153Q02HA', 'CURIOAGR', 'ST261Q01JA', 'ICTINFO', 'ST301Q01JA', 'PA188Q05JA', 'ST322Q04JA', 'ST268Q09JA', 'ST266Q05JA', 'WB163Q01HA', 'CREATHME', 'PSYCHSYM', 'ICTENQ', 'PA007Q01TA', 'IC178Q01JA', 'ST267Q01JA', 'ST291Q09JA', 'ST226Q01JA', 'ASSERAGR', 'IC183Q15JA', 'PASCHPOL', 'FL169Q02HA', 'BFMJ2', 'ST305Q10JA', 'ST324Q10JA', 'WB163Q08HA', 'WB031Q01NA', 'CREATAS', 'ST260Q01JA', 'MATHMOT', 'ST309Q01JA', 'LIFESAT', 'BSMJ', 'ST301Q02JA', 'ST303Q05JA', 'PA166Q01HA', 'GROSAGR', 'ST322Q06JA', 'FL164Q14HA', 'ST345Q08JA', 'WB153Q04HA', 'EXERPRAC', 'ICTSCH', 'FL163Q04HA', 'TEACHSUP', 'ST268Q02JA', 'ST307Q01JA', 'ICTFEED', 'ST330Q04WA', 'PA167Q02HA', 'ST307Q06JA', 'FL159Q01HA', 'CYC', 'ST008Q03JA', 'ICTRES', 'MISSSC', 'ST335Q07JA', 'ST352Q04JA', 'ATTCNCFM', 'IC180Q05JA', 'WB173Q02HA', 'IC184Q01JA', 'IC173Q02JA', 'ST352Q03JA', 'PA007Q04TA', 'WB154Q05HA', 'ST290Q09WA', 'PA185Q08JA', 'PA006Q04TA', 'PA003Q19WA', 'PA177Q01HA', 'ST289Q10WA', 'ST256Q09JA', 'IC183Q08JA', 'ST254Q04JA', 'PA187Q04JA', 'ST311Q10JA', 'ST267Q06JA', 'IC174Q06JA', 'WB177Q03HA', 'ST330Q03WA', 'FL169Q11JA', 'ST309Q05JA', 'ST285Q09JA', 'WB032Q01NA', 'ST345Q04JA', 'ST038Q07NA', 'ST355Q08JA', 'ST275Q05WA', 'IC181Q04JA', 'ACCESSFA', 'PA187Q06JA', 'FL171Q07JA', 'ST345Q02JA', 'FL164Q12HA', 'PA006Q14HA', 'HISEI', 'PA008Q08NA', 'CREATACT', 'IC170Q02JA', 'ST309Q06JA', 'ST283Q08JA', 'GRADE', 'FL171Q11JA', 'ST006Q04JA', 'MATHEF21', 'IC174Q02JA', 'PA003Q14JA', 'ST337Q06JA', 'ST253Q01JA', 'PA009Q03NA', 'ST305Q04JA', 'ST300Q08JA', 'ST309Q04JA', 'FL167Q01HA', 'ICTOUT', 'IC178Q05JA', 'BODYIMA', 'ST251Q01JA', 'ST327Q08JA', 'ST311Q02JA', 'ST038Q08NA', 'ST293Q07JA', 'ST263Q06JA', 'IC175Q01JA', 'ST330Q02WA', 'ST003D03T', 'ST313Q02JA', 'ST340Q03JA', 'ST326Q02JA', 'ST335Q02JA', 'ST345Q10JA', 'PA006Q13HA', 'ST315Q09JA', 'FEELSAFE', 'FL150Q01TA', 'ST307Q07JA', 'ST313Q04JA', 'ST330Q01WA', 'IC172Q09JA', 'ST300Q10JA', 'ST348Q01JA', 'ST285Q06JA', 'ST337Q03JA', 'FL160Q01HA', 'ST326Q07JA', 'IC170Q03JA', 'FL162Q05HA', 'ST256Q07JA', 'FL169Q07JA', 'ST311Q04JA', 'ST285Q08JA', 'WB163Q04HA', 'ST273Q06JA', 'ST291Q10JA', 'ST268Q04JA', 'PA018Q01NA', 'ST345Q03JA', 'FAMCON', 'IC175Q05JA', 'WB178Q06HA', 'ST303Q07JA', 'PA003Q20WA', 'ST305Q07JA', 'ST324Q11JA', 'ST326Q03JA', 'ST342Q07JA', 'ST311Q01JA', 'OPENART', 'ST291Q04JA', 'TARDYSD', 'PA175Q03JA', 'WB161Q01HA', 'WB152Q01HA', 'CURSUPP', 'WB173Q04HA', 'PA177Q06HA', 'FL160Q03HA', 'ST331Q03JA', 'IC172Q03JA', 'ST355Q03JA', 'PA003Q13JA', 'FL174Q01JA', 'ST293Q02JA', 'PA003Q17JA', 'FL161Q03HA', 'FL169Q10JA', 'ST353Q06JA', 'ST285Q07JA', 'ST351Q06JA', 'IC177Q04JA', 'PA008Q07NA', 'ST038Q11JA', 'ST300Q03JA', 'ST004D01T', 'REPEAT', 'ST327Q06JA', 'ST283Q01JA', 'IC177Q03JA', 'ST337Q07JA', 'ST256Q02JA', 'ST261Q05JA', 'ST309Q03JA', 'ST251Q03JA', 'WB163Q02HA', 'MATHPREF', 'PA003Q12JA', 'ST305Q09JA', 'IC176Q03JA', 'PA007Q15NA', 'FL164Q16HA', 'FL170Q07JA', 'PA186Q03JA', 'ST354Q05JA', 'ST351Q01JA', 'ST034Q02TA', 'ST275Q03WA', 'PA185Q05JA', 'ST291Q01JA', 'ST254Q03JA', 'ST337Q05JA', 'WB178Q01HA', 'IC177Q05JA', 'ST301Q09JA', 'PA042Q01TA', 'FL164Q02HA', 'ST342Q01JA', 'ST327Q07JA', 'PA183Q08JA', 'ST326Q11JA', 'ST334Q04JA', 'PA009Q06NA', 'ST336Q01JA', 'ST334Q07JA', 'FL170Q01JA', 'PA003Q01TA', 'ST315Q10JA', 'ST285Q03JA', 'IC171Q02JA', 'ST021Q01TA', 'ST294Q04JA', 'ST127Q02TA', 'WB172Q01HA', 'IMMIG', 'FL166Q06HA', 'ST322Q07JA', 'ST276Q06JA', 'ST261Q04JA', 'EFFORT2', 'WB171Q03HA', 'ST313Q03JA', 'ST022Q01TA', 'ST250Q02JA', 'ST268Q05JA', 'DURECEC', 'PA009Q10NA', 'PA008Q03TA', 'STRESAGR', 'PA188Q02JA', 'FL174Q05JA', 'ST006Q01JA', 'ST348Q03JA', 'ST311Q05JA', 'PA007Q12NA', 'ST294Q05JA', 'FLCONFIN', 'IC183Q13JA', 'WB155Q05HA', 'FL162Q01HA', 'WB155Q01HA', 'ST267Q03JA', 'FL169Q01HA', 'ST268Q06JA', 'ST315Q08JA', 'ST336Q05JA', 'ST355Q04JA', 'ST300Q01JA', 'ST324Q12JA', 'ST301Q03JA', 'FL150Q02TA', 'FL169Q03HA', 'IC172Q08JA', 'FL160Q04HA', 'PA188Q04JA', 'FL171Q03JA', 'WB166Q04HA', 'ST261Q08JA', 'PA008Q02TA', 'ST260Q02JA', 'ST289Q09WA', 'FL170Q05JA', 'IC174Q05JA', 'WB178Q02HA', 'INFOSEEK', 'FL167Q07JA', 'FL159Q04HA', 'PA003Q16JA', 'ST337Q08JA', 'ST297Q09JA', 'WB032Q02NA', 'FRINFLFM', 'PA007Q03TA', 'FL164Q03HA', 'ST301Q06JA', 'WB154Q08HA', 'IC177Q02JA', 'ST296Q04JA', 'FL162Q06HA', 'IC183Q16JA', 'ST263Q02JA', 'ST291Q06JA', 'ST259Q01JA', 'WB153Q05HA', 'IC183Q10JA', 'IC174Q09JA', 'WB163Q06HA', 'ST008Q05JA', 'FL164Q13HA', 'WB154Q07HA', 'COOPAGR', 'ST309Q08JA'}

	variable	count_csv	unique_count_parquet	null_count_parquet	final_type
87	VER_DAT	1	281	0	char
1148	CNT	82	80	0	char
1150	NatCen	85	84	0	char
1151	SUBNATIO	93	88	0	char
1153	ST250D07JA	110	106	0	char
1154	ST250D06JA	130	126	0	char
1155	ST330D10WA	145	143	0	char
1157	ST251D09JA	172	166	0	char
1158	ST251D08JA	220	214	0	char
1159	COBN_S	253	245	0	char
1160	COBN_M	253	245	0	char
1161	COBN_F	253	246	0	char
1163	OCOD1	625	624	0	char
1164	OCOD2	625	624	0	char
1165	OCOD3	625	607	0	char
1166	PROGN	664	438	0	char
1167	STRATUM	1318	1014	0	char

Character columns: {'COBN_S', 'COBN_M', 'OCOD2', 'OCOD3', 'SUBNATIO', 'PROGN', 'STRATUM', 'ST250D06JA', 'ST251D08JA', 'VER_DAT', 'ST250D07JA', 'COBN_F', 'ST251D09JA', 'OCOD1', 'ST330D10WA', 'NatCen', 'CNT'}

	variable	count_csv	unique_count_parquet	null_count_parquet	final_type
88	OECD	3	2	0	categorical
89	ADMINMODE	3	2	0	categorical
90	Option_CT	3	2	0	categorical
91	Option_FL	3	2	0	categorical
92	Option_ICTQ	3	2	0	categorical
93	Option_WBQ	3	2	0	categorical
94	Option_PQ	3	2	0	categorical
95	Option_TQ	3	2	0	categorical
96	Option_UH	3	2	0	categorical
97	UNIT	4	3	0	categorical
1019	ST001D01T	12	10	0	categorical
1145	LANGTEST_COG	72	58	0	categorical
1149	CNTRYID	82	80	0	categorical
1152	BOOKID	106	97	0	categorical
1156	REGION	146	138	0	categorical
1162	LANGN	265	239	0	categorical

Categorical columns: {'LANGN', 'UNIT', 'Option_FL', 'ADMINMODE', 'LANGTEST_COG', 'Option_ICTQ', 'Option_CT', 'Option_WBQ', 'Option_TQ', 'ST001D01T', 'REGION', 'BOOKID', 'Option_PQ', 'Option_UH', 'CNTRYID', 'OECD'}

```
In [12]: # drop columns that are constant
features_df = final_df.copy()
features_df = features_df[~features_df['variable'].isin(columns_to_drop)]

# change final_type for categorical exceptions to numerical
features_df.loc[features_df['variable'].isin(cat_exceptions), 'final_type'] = 'numerical'

# change final_type for categorical and char columns to categorical
```

```
features_df.loc[features_df['variable'].isin(categorical_columns), 'final_type'] = 'categorical'
features_df.loc[features_df['variable'].isin(char_columns), 'final_type'] = 'categorical'
```

```
features_df
```

Out[12]:

	variable	count_csv	unique_count_parquet	null_count_parquet	final_type
0	CNTSCHID	1	21567	0	numerical
1	CNTSTUID	1	490995	0	numerical
3	AGE	1	16	0	numerical
4	W_FSTUWT	1	45875	0	numerical
5	W_FSTURWT1	1	56884	0	numerical
...	...	...	...	...	...
1163	OCOD1	625	624	0	categorical
1164	OCOD2	625	624	0	categorical
1165	OCOD3	625	607	0	categorical
1166	PROGN	664	438	0	categorical
1167	STRATUM	1318	1014	0	categorical

119 rows × 5 columns

Categorical variables are encoded according to their cardinality. Variables with fewer than 20 categories are transformed using one-hot encoding, whereas higher-cardinality categorical variables are transformed using frequency encoding to avoid excessive dimensionality.

In [13]:

```
features_df['encoding'] = features_df['final_type']

features_df.loc[(features_df['final_type'] == 'categorical') & (features_df['unique_count_parquet'] < 20), 'encoding'] = 'one-hot'
features_df.loc[(features_df['final_type'] == 'categorical') & (features_df['unique_count_parquet'] >= 20), 'encoding'] = 'frequency'

features_df.drop(columns=['count_csv', 'unique_count_parquet', 'null_count_parquet', 'final_type'], inplace=True)
features_df.to_csv('features_encoding.csv', index=False)
features_df
```

Out[13]:

	variable	encoding
0	CNTSCHID	numerical
1	CNTSTUID	numerical
3	AGE	numerical
4	W_FSTUWT	numerical
5	W_FSTURWT1	numerical
...	...	...
1163	OCOD1	frequency
1164	OCOD2	frequency
1165	OCOD3	frequency
1166	PROGN	frequency
1167	STRATUM	frequency

119 rows × 2 columns

In []:

```
import pandas as pd

features_df = pd.read_csv('features_encoding.csv')
target = 'PV_MATH_MEAN'

train_df = pd.read_parquet('data/train_transformed.parquet')
test_df = pd.read_parquet('data/test_transformed.parquet')

encoding_map = features_df.set_index('variable')['encoding'].to_dict()
selected_features = set(features_df['variable'])

train_X = train_df[[c for c in train_df.columns if c in selected_features]].copy()
```

```

test_X = test_df[[c for c in test_df.columns if c in selected_features]].copy()

numerical_cols = [c for c in train_X.columns if encoding_map.get(c) == 'numerical']
frequency_cols = [c for c in train_X.columns if encoding_map.get(c) == 'frequency']
one_hot_cols = [c for c in train_X.columns if encoding_map.get(c) == 'one-hot']

freq_maps = {
    col: train_X[col].fillna('__MISSING__').value_counts(normalize=True)
    for col in frequency_cols
}

one_hot_categories = {
    col: train_X[col].dropna().unique().tolist()
    for col in one_hot_cols
}

def encode_dataset(df):
    y = df[target] if target in df.columns else None
    X = df.drop(columns=[target], errors='ignore')
    X = X[[c for c in X.columns if c in selected_features]].copy()

    parts = []

    if numerical_cols:
        parts.append(X[numerical_cols])

    if frequency_cols:
        freq_encoded = pd.DataFrame(index=X.index)
        for col in frequency_cols:
            freq_encoded[col] = X[col].fillna('__MISSING__').map(freq_maps[col]).fillna(0.0)
        parts.append(freq_encoded)

    if one_hot_cols:
        onehot_parts = []
        for col in one_hot_cols:
            cat = pd.Categorical(X[col], categories=one_hot_categories[col])
            dummies = pd.get_dummies(cat, prefix=col, dtype=int)
            expected_cols = [f"{col}_{v}" for v in one_hot_categories[col]]
            dummies = dummies.reindex(columns=expected_cols, fill_value=0)
            onehot_parts.append(dummies)
        parts.append(pd.concat(onehot_parts, axis=1) if onehot_parts else pd.DataFrame(index=X.index))

    encoded = pd.concat(parts, axis=1)

    if y is not None:
        encoded[target] = y.values

    return encoded

train_encoded = encode_dataset(train_df)
test_encoded = encode_dataset(test_df)

train_encoded.to_parquet('data/train_encoded.parquet', index=False)
test_encoded.to_parquet('data/test_encoded.parquet', index=False)

print(train_encoded.shape)
print(test_encoded.shape)

```

(490995, 140)

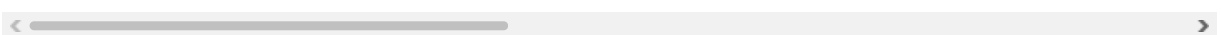
(122749, 140)

In [51]: train_encoded

Out[51]:

	CNTSCHID	CNTSTUID	AGE	W_FSTUWT	W_FSTURWT1	W_FSTURWT2	W_FSTURWT3	W_FSTURWT4	W_FSTURWT5
0	9600018.0	9601263.0	15.50	1.04444	0.53403	1.60129	1.56678	1.58413	1.60129
1	3100178.0	3108247.0	16.25	5.80838	8.84733	2.85278	2.85278	8.84733	8.84733
2	30000029.0	30002369.0	15.92	14.87932	21.58813	21.11367	7.58124	7.92197	7.75119
3	15200066.0	15206832.0	16.00	31.21619	15.60810	15.60810	46.82429	46.82429	15.60810
4	12400503.0	12421460.0	16.08	25.78511	14.60340	42.43552	40.37669	39.75425	13.48119
...	...	...	...	...	...	...	...	...	...
490990	15200101.0	15206853.0	15.92	55.95034	83.92551	33.16322	78.13289	33.16322	78.13289
490991	37600176.0	37604707.0	15.58	17.10418	8.55209	25.65626	25.65626	25.65626	25.65626
490992	49800193.0	49803806.0	15.58	4.32762	6.49143	6.49143	6.49143	6.49143	6.49143
490993	19100119.0	19101221.0	15.75	4.71078	7.06617	7.06617	2.35539	7.06617	2.35539
490994	17000207.0	17005658.0	15.92	61.38186	92.07279	30.69093	92.07279	92.07279	92.07279

490995 rows × 140 columns



Mutual information is used to select the most relevant encoded predictors. To make the computation feasible on a large dataset, the score is estimated using a reproducible sample of 60,000 training rows. The test set is not used during feature selection, which prevents data leakage.

In [54]:

```
import numpy as np
import pandas as pd
from sklearn.feature_selection import mutual_info_regression
from sklearn.impute import SimpleImputer

RANDOM_STATE = 42
N_FEATURES_TO_SELECT = 15
MAX_ROWS_FOR_SELECTION = 60_000
target = 'PV_MATH_MEAN'

train_encoded = pd.read_parquet('data/train_encoded.parquet')
test_encoded = pd.read_parquet('data/test_encoded.parquet')

X_train = train_encoded.drop(columns=[target])
y_train = train_encoded[target]
X_test = test_encoded.reindex(columns=X_train.columns.tolist() + [target])

idx = X_train.sample(min(MAX_ROWS_FOR_SELECTION, len(X_train)), random_state=RANDOM_STATE).index
X_sel = X_train.loc[idx].apply(pd.to_numeric, errors='coerce')
y_sel = y_train.loc[idx]

X_sel = SimpleImputer(strategy='median').fit_transform(X_sel)
try:
    mi_scores = mutual_info_regression(X_sel, y_sel, n_neighbors=3, random_state=RANDOM_STATE, n_jobs=-1)
except TypeError: # compatibility with older versions of scikit-learn
    mi_scores = mutual_info_regression(X_sel, y_sel, n_neighbors=3, random_state=RANDOM_STATE)

feature_selection_scores = (
    pd.DataFrame({'feature': X_train.columns, 'mutual_info_score': mi_scores})
    .replace([np.inf, -np.inf], np.nan)
    .fillna({'mutual_info_score': 0})
    .sort_values('mutual_info_score', ascending=False)
)

selected_feature_names = feature_selection_scores.head(
    min(N_FEATURES_TO_SELECT, X_train.shape[1])
)['feature'].tolist()

train_selected = train_encoded[selected_feature_names + [target]].copy()
test_selected = test_encoded[selected_feature_names + [target]].copy()

train_selected.to_parquet('data/train_selected_features.parquet', index=False)
test_selected.to_parquet('data/test_selected_features.parquet', index=False)
feature_selection_scores.to_csv('feature_selection_scores.csv', index=False)

print(f'Rows used for selection: {len(idx)},')
print(f'Total features before selection: {X_train.shape[1]}')
```

```
print(f'Features selected: {len(selected_feature_names)}')
display(feature_selection_scores.head(20))
```

Rows used for selection: 60,000
 Total features before selection: 139
 Features selected: 15

	feature	mutual_info_score
0	CNTSCHID	0.190830
1	CNTSTUID	0.190474
104	COBN_M	0.166780
103	COBN_S	0.152854
107	VER_DAT	0.149563
91	REGION	0.144366
102	PROGN	0.137873
88	NatCen	0.134771
90	SUBNATIO	0.131613
87	CNTRYID	0.130865
105	COBN_F	0.130428
86	CNT	0.130202
92	LANGTEST_COG	0.105479
89	STRATUM	0.105192
106	LANGN	0.097789
94	ST250D06JA	0.070309
109	OECD_1.0	0.065569
108	OECD_0.0	0.065569
99	OCOD1	0.062101
95	ST250D07JA	0.046729

5. Exploratory Data Analysis

This section summarizes the selected predictors and the target variable. The analysis is intentionally focused on the reduced feature set because the encoded dataset is high-dimensional and expensive to visualize directly.

```
In [1]: import matplotlib.pyplot as plt
import pandas as pd

TARGET = 'PV_MATH_MEAN'
train_selected = pd.read_parquet('data/train_selected_features.parquet')
test_selected = pd.read_parquet('data/test_selected_features.parquet')

print('Training shape:', train_selected.shape)
print('Test shape:', test_selected.shape)
display(train_selected.describe().T)

plt.figure(figsize=(7, 4))
train_selected[TARGET].hist(bins=40)
plt.title('Distribution of the Target Variable')
plt.xlabel(TARGET)
plt.ylabel('Frequency')
plt.show()

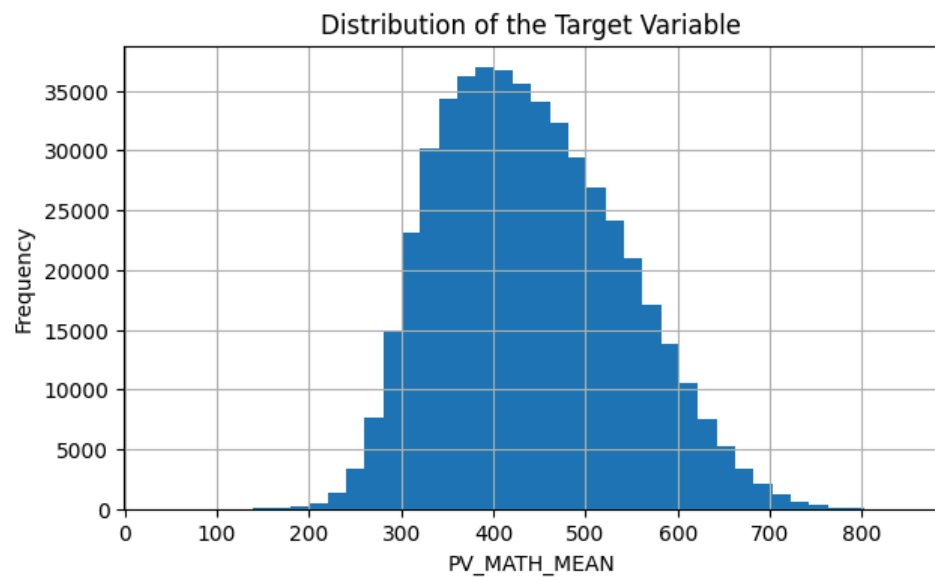
corr = train_selected.corr(numeric_only=True)[TARGET].drop(TARGET).sort_values(key=lambda s: s.abs(), ascending=False)
display(corr.to_frame('correlation_with_target'))

plt.figure(figsize=(7, 4))
corr.head(15).sort_values().plot(kind='barh')
plt.title('Top Selected Features by Absolute Correlation with the Target')
plt.xlabel('Pearson correlation')
plt.ylabel('Feature')
plt.show()
```

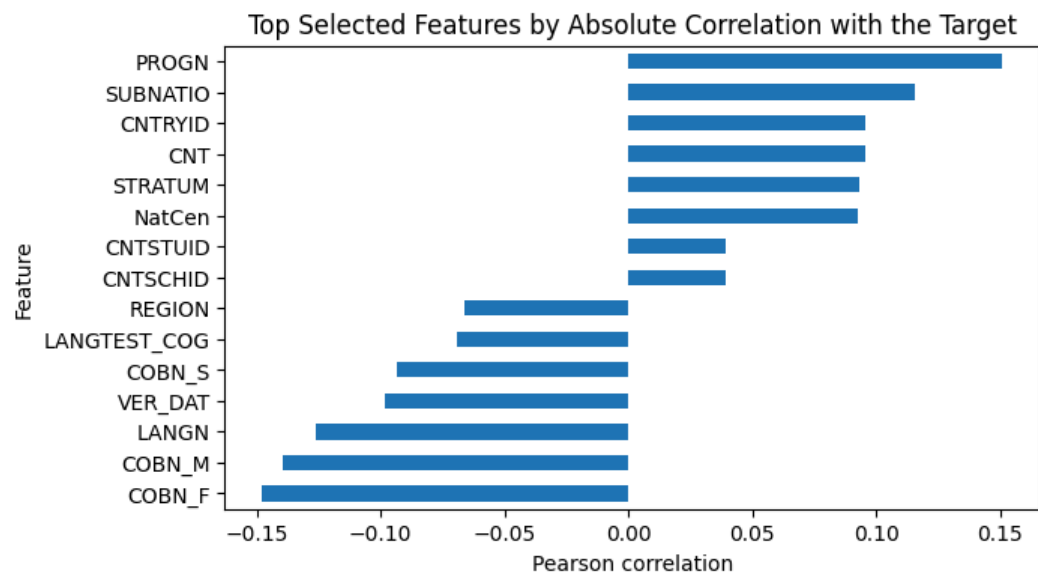
Training shape: (490995, 16)

Test shape: (122749, 16)

	count	mean	std	min	25%	50%	75%	r
CNTSCHID	490995.0	4.347535e+07	2.558766e+07	800001.000000	2.140006e+07	3.980054e+07	6.880013e+07	9.010018e
CNTSTUID	490995.0	4.341995e+07	2.548673e+07	800002.000000	2.140192e+07	3.981965e+07	6.880513e+07	8.600749e
COBN_M	490995.0	1.173892e-02	9.444047e-03	0.000006	7.128382e-03	9.586656e-03	1.259891e-02	4.621228e
COBN_S	490995.0	1.266117e-02	8.912997e-03	0.000004	8.533692e-03	1.014063e-02	1.325675e-02	4.203505e
VER_DAT	490995.0	4.500475e-03	1.573780e-03	0.000029	3.629365e-03	4.592715e-03	5.560138e-03	9.959368e
REGION	490995.0	1.124339e-02	7.370658e-03	0.000442	8.238373e-03	1.026283e-02	1.181478e-02	4.010224e
PROGN	490995.0	8.789047e-03	8.612809e-03	0.000002	3.503091e-03	6.596809e-03	1.031375e-02	4.009613e
NatCen	490995.0	1.542282e-02	1.060025e-02	0.005114	1.001029e-02	1.115083e-02	1.670282e-02	5.016548e
SUBNATIO	490995.0	1.495975e-02	1.065817e-02	0.003631	9.888084e-03	1.097160e-02	1.373537e-02	5.016548e
CNTRYID	490995.0	1.666470e-02	1.154059e-02	0.005114	1.003065e-02	1.128728e-02	1.761118e-02	5.016548e
COBN_F	490995.0	1.195872e-02	1.027886e-02	0.000004	7.417591e-03	9.521482e-03	1.255614e-02	4.993330e
CNT	490995.0	1.666470e-02	1.154059e-02	0.005114	1.003065e-02	1.128728e-02	1.761118e-02	5.016548e
LANGTEST_COG	490995.0	7.359493e-02	7.364055e-02	0.000026	1.133616e-02	2.856445e-02	1.726800e-01	1.736657e
STRATUM	490995.0	5.033530e-03	7.450498e-03	0.000002	1.107954e-03	2.244422e-03	6.348333e-03	3.760934e
LANGN	490995.0	4.939093e-02	5.707274e-02	0.000002	9.560179e-03	1.701239e-02	9.866903e-02	1.618998e
PV_MATH_MEAN	490995.0	4.409297e+02	9.842185e+01	39.171300	3.648105e+02	4.324173e+02	5.098623e+02	8.432881e



	correlation_with_target
PROGN	0.150943
COBN_F	-0.147918
COBN_M	-0.139508
LANGN	-0.125959
SUBNATIO	0.115614
VER_DAT	-0.098325
CNTRYID	0.095655
CNT	0.095655
COBN_S	-0.093590
STRATUM	0.092925
NatCen	0.092565
LANGTEST_COG	-0.069368
REGION	-0.066005
CNTSTUID	0.039224
CNTSCHID	0.039146



The target variable exhibits an approximately normal distribution with sufficient variability for regression modeling. The correlation analysis shows that no individual feature has a strong linear relationship with `PV_MATH_MEAN` (since $|r| < 0.16$), suggesting that mathematical performance is influenced by multiple interacting factors rather than a single predictor. These findings support the use of non-linear models, such as Random Forest and Neural Networks, to capture more complex relationships within the data.

The selected features mainly represent demographic, geographic, linguistic, and institutional characteristics, including country, region, school, language, and socioeconomic indicators. Although each variable individually exhibits only a weak correlation with the target, together they capture complementary information that is likely to improve predictive performance. This supports the feature selection approach, as machine learning models can exploit complex interactions among these variables that are not evident through pairwise linear correlations alone.

6. t-SNE

A two-dimensional t-SNE projection is used as an exploratory visualization of the selected feature space. The points are colored according to the target value, allowing visual inspection of whether students with similar mathematics performance occupy nearby regions in the learned embedding. To keep the computation efficient, the visualization is based on a reproducible sample.

```
In [3]: from sklearn.manifold import TSNE
        from sklearn.preprocessing import StandardScaler
```

```

from sklearn.impute import SimpleImputer
import matplotlib.pyplot as plt
import pandas as pd

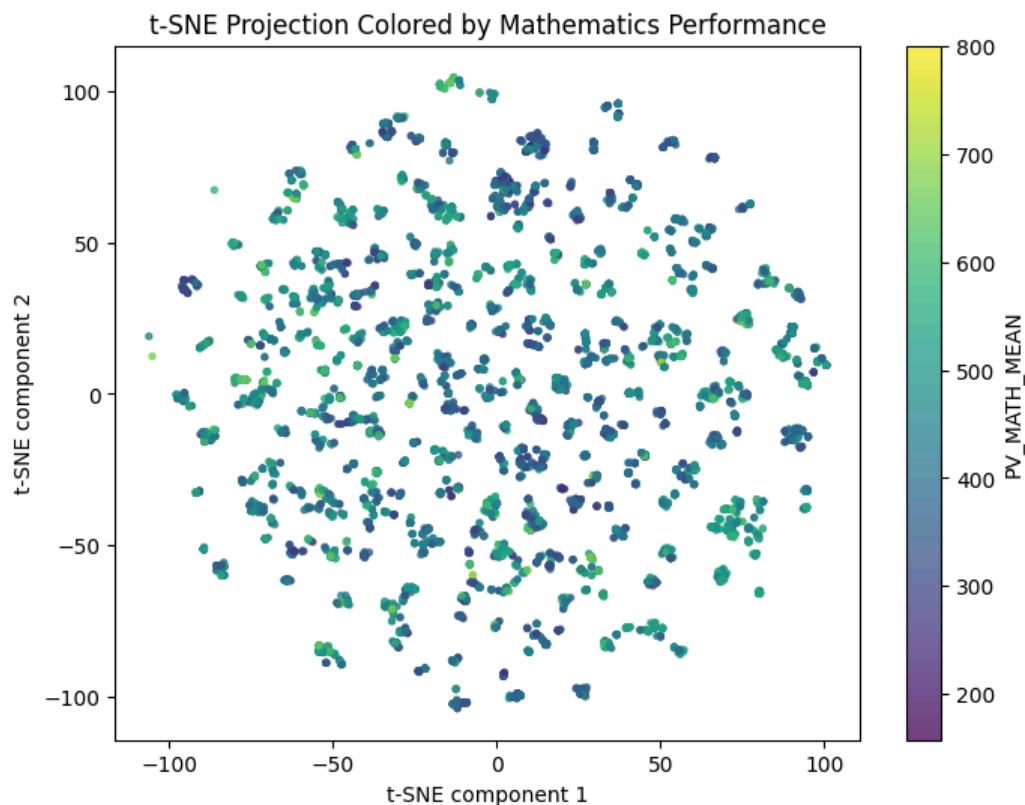
TARGET = 'PV_MATH_MEAN'
TSNE_SAMPLE_SIZE = 10_000
RANDOM_STATE = 42

train_selected = pd.read_parquet('data/train_selected_features.parquet')
sample = train_selected.sample(min(TSNE_SAMPLE_SIZE, len(train_selected)), random_state=RANDOM_STATE)
X_tsne = SimpleImputer(strategy='median').fit_transform(sample.drop(columns=[TARGET]))
X_tsne = StandardScaler().fit_transform(X_tsne)

embedding = TSNE(n_components=2, perplexity=30, learning_rate='auto', init='pca', random_state=RANDOM_STATE).fit_

plt.figure(figsize=(8, 6))
scatter = plt.scatter(embedding[:, 0], embedding[:, 1], c=sample[TARGET], s=8, alpha=0.75)
plt.colorbar(scatter, label=TARGET)
plt.title('t-SNE Projection Colored by Mathematics Performance')
plt.xlabel('t-SNE component 1')
plt.ylabel('t-SNE component 2')
plt.show()

```



The t-SNE projection shows that the observations form several local clusters, but the target values are not clearly separated into distinct regions. Instead, low, medium, and high mathematics performance values appear partially mixed across the two-dimensional space. This suggests that the selected features contain some local structure, but the relationship with `PV_MATH_MEAN` is complex and not easily captured by a simple visual separation. Therefore, non-linear supervised models may still be useful, although strong class-like clustering should not be expected from these features alone.

7. Pipelines

Two supervised regression pipelines are evaluated: a Random Forest Regressor and a neural network implemented as a multilayer perceptron. Both pipelines include median imputation. The neural network pipeline also includes standardization because gradient-based models are sensitive to feature scale.

```

In [ ]: import pandas as pd
from sklearn.ensemble import RandomForestRegressor
from sklearn.neural_network import MLPRegressor
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split

```



```

TARGET = 'PV_MATH_MEAN'
RANDOM_STATE = 42

train_selected = pd.read_parquet('data/train_selected_features.parquet')
test_selected = pd.read_parquet('data/test_selected_features.parquet')

X = train_selected.drop(columns=[TARGET])
y = train_selected[TARGET]
X_test = test_selected.drop(columns=[TARGET])
y_test = test_selected[TARGET]

X_train_model, X_valid_model, y_train_model, y_valid_model = train_test_split(X, y, test_size=0.2, random_state=1)

rf_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('model', RandomForestRegressor(n_estimators=200, random_state=RANDOM_STATE, n_jobs=-1))
])
nn_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler()),
    ('model', MLPRegressor(hidden_layer_sizes=(64, 32), max_iter=250, early_stopping=True, random_state=RANDOM_STATE))
])

```

```

In [2]: from sklearn.metrics import root_mean_squared_error as rmse, mean_absolute_error, r2_score

def regression_metrics(model, X_eval, y_eval):
    pred = model.predict(X_eval)
    return {'RMSE': rmse(y_eval, pred), 'MAE': mean_absolute_error(y_eval, pred), 'R2': r2_score(y_eval, pred)}

baseline_results = []
for name, model in {'Random Forest': rf_pipeline, 'Neural Network': nn_pipeline}.items():
    model.fit(X_train_model, y_train_model)
    baseline_results.append({'model': name, **regression_metrics(model, X_valid_model, y_valid_model)})

display(pd.DataFrame(baseline_results).sort_values('RMSE'))

```

	model	RMSE	MAE	R2
0	Random Forest	72.060463	56.521425	0.462687
1	Neural Network	75.795248	60.338985	0.405548

8. Hyperparameter Optimization

A compact grid search is used to tune one primary hyperparameter for each model while minimizing cross-validated RMSE. For the Random Forest model, the search focuses on tree depth, whereas for the Neural Network it focuses on the L2 regularization parameter. The remaining hyperparameters are intentionally restricted to keep the search computationally feasible while still producing interpretable optimization curves.

```

In [2]: import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import GridSearchCV, KFold
from sklearn.metrics import mean_squared_error, make_scorer
from sklearn.ensemble import RandomForestRegressor
from sklearn.neural_network import MLPRegressor
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer

RANDOM_STATE = 42
target = "PV_MATH_MEAN"

OPTIMIZATION_SAMPLE_SIZE = 30_000
CV_FOLDS_OPTIMIZATION = 5

train_selected = pd.read_parquet("data/train_selected_features.parquet")
test_selected = pd.read_parquet("data/test_selected_features.parquet")

X_train = train_selected.drop(columns=[target])
y_train = train_selected[target]

if len(X_train) > OPTIMIZATION_SAMPLE_SIZE:
    opt_idx = X_train.sample(

```

```

        n=OPTIMIZATION_SAMPLE_SIZE,
        random_state=RANDOM_STATE
    ).index
    X_opt = X_train.loc[opt_idx]
    y_opt = y_train.loc[opt_idx]
else:
    X_opt = X_train
    y_opt = y_train

print(f"Optimization sample size: {X_opt.shape[0]:,} rows")
print(f"Number of features: {X_opt.shape[1]}")
print(f"Optimization CV: {CV_FOLDS_OPTIMIZATION}-fold")
print("Starting compact grid search optimization...\n")

rmse_scorer = make_scorer(
    lambda y_true, y_pred: np.sqrt(mean_squared_error(y_true, y_pred)),
    greater_is_better=False
)

cv_opt = KFold(
    n_splits=CV_FOLDS_OPTIMIZATION,
    shuffle=True,
    random_state=RANDOM_STATE
)

rf_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("model", RandomForestRegressor(
        n_jobs=-1,
        random_state=RANDOM_STATE
    ))
])

nn_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
    ("model", MLPRegressor(
        early_stopping=True,
        validation_fraction=0.15,
        n_iter_no_change=8,
        max_iter=300,
        random_state=RANDOM_STATE
    ))
])

search_spaces = {
    "Random Forest": {
        "pipeline": rf_pipeline,
        "params": {
            # Main hyperparameter for the optimization plot
            "model__max_depth": [10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30],

            # Limited secondary hyperparameters to control runtime
            "model__n_estimators": [120],
            "model__min_samples_leaf": [20],
            "model__max_features": ["sqrt"]
        }
    },
    "Neural Network": {
        "pipeline": nn_pipeline,
        "params": {
            # Main hyperparameter for the optimization plot
            "model__alpha": [1e-4, 1e-2, 1e-1, 1e0, 1e+1, 1e+2],

            # Limited secondary hyperparameters to control runtime
            "model__hidden_layer_sizes": [(64, 64)],
            "model__learning_rate_init": [1e-3],
            "model__batch_size": [512]
        }
    }
}

optimization_results = {}
optimization_summary = []

for model_name, config in search_spaces.items():
    print("=" * 70)
    print(f"Optimizing: {model_name}")

    n_combinations = np.prod([len(values) for values in config["params"].values()])

```

```

total_fits = n_combinations * CV_FOLDS_OPTIMIZATION

print(f"Grid combinations: {n_combinations}")
print(f"Total CV fits: {total_fits}")
print("Running GridSearchCV...\n")

start_time = time.time()

search = GridSearchCV(
    estimator=config["pipeline"],
    param_grid=config["params"],
    scoring=rmse_scorer,
    cv=cv_opt,
    n_jobs=-1,
    return_train_score=True,
    verbose=2
)

search.fit(X_opt, y_opt)

elapsed = time.time() - start_time
best_rmse = -search.best_score_

print(f"\nFinished {model_name}")
print(f"Elapsed time: {elapsed / 60:.2f} minutes")
print(f"Best CV RMSE: {best_rmse:.4f}")
print(f"Best parameters: {search.best_params_}\n")

cv_results = pd.DataFrame(search.cv_results_)
cv_results["model_name"] = model_name
cv_results["mean_train_rmse"] = -cv_results["mean_train_score"]
cv_results["mean_cv_rmse"] = -cv_results["mean_test_score"]

optimization_results[model_name] = {
    "search": search,
    "cv_results": cv_results,
    "best_estimator": search.best_estimator_,
    "best_params": search.best_params_,
    "best_cv_rmse": best_rmse
}

optimization_summary.append({
    "model": model_name,
    "best_cv_rmse": best_rmse,
    "elapsed_minutes": elapsed / 60,
    "grid_combinations": n_combinations,
    "total_cv_fits": total_fits,
    "best_params": search.best_params_
})

optimization_summary = pd.DataFrame(optimization_summary).sort_values("best_cv_rmse")

optimization_cv_results = pd.concat(
    [result["cv_results"] for result in optimization_results.values()],
    ignore_index=True
)

best_model_name = optimization_summary.iloc[0]["model"]
best_model = optimization_results[best_model_name]["best_estimator"]
best_params = optimization_results[best_model_name]["best_params"]

best_rf_model = optimization_results["Random Forest"]["best_estimator"]
best_nn_model = optimization_results["Neural Network"]["best_estimator"]

print("=" * 70)
print("Optimization completed.")
print(f"Best model: {best_model_name}")
print(f"Best CV RMSE: {optimization_summary.iloc[0]['best_cv_rmse']:.4f}")
print(f"Best parameters: {best_params}")

display(optimization_summary)

```

Optimization sample size: 30,000 rows
 Number of features: 15
 Optimization CV: 5-fold
 Starting compact grid search optimization...

=====
 Optimizing: Random Forest
 Grid combinations: 11
 Total CV fits: 55
 Running GridSearchCV...

Fitting 5 folds for each of 11 candidates, totalling 55 fits

Finished Random Forest
 Elapsed time: 0.29 minutes
 Best CV RMSE: 75.1098
 Best parameters: {'model__max_depth': 28, 'model__max_features': 'sqrt', 'model__min_samples_leaf': 20, 'model__n_estimators': 120}

=====
 Optimizing: Neural Network
 Grid combinations: 6
 Total CV fits: 30
 Running GridSearchCV...

Fitting 5 folds for each of 6 candidates, totalling 30 fits

Finished Neural Network
 Elapsed time: 2.21 minutes
 Best CV RMSE: 82.2280
 Best parameters: {'model__alpha': 10.0, 'model__batch_size': 512, 'model__hidden_layer_sizes': (64, 64), 'model__learning_rate_init': 0.001}

=====
 Optimization completed.
 Best model: Random Forest
 Best CV RMSE: 75.1098
 Best parameters: {'model__max_depth': 28, 'model__max_features': 'sqrt', 'model__min_samples_leaf': 20, 'model__n_estimators': 120}

c:\Users\nicoc\AppData\Local\Programs\Python\Python310\lib\site-packages\sklearn\normal_network_multilayer_perceptron.py:781: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (300) reached and the optimization has n't converged yet.
 warnings.warn(

	model	best_cv_rmse	elapsed_minutes	grid_combinations	total_cv_fits	best_params
0	Random Forest	75.109764	0.290885	11	55	{'model__max_depth': 28, 'model__max_features': ...}
1	Neural Network	82.227965	2.206001	6	30	{'model__alpha': 10.0, 'model__batch_size': 51...

```
In [3]: import joblib
import os

os.makedirs("models", exist_ok=True)

joblib.dump(best_rf_model, "models/best_rf_model.pkl")
joblib.dump(best_nn_model, "models/best_nn_model.pkl")
```

Out[3]: ['models/best_nn_model.pkl']

```
In [37]: import matplotlib.pyplot as plt
import pandas as pd

def plot_hyperparameter_vs_loss(model_name, hyperparameter, log_scale=False):
    param_col = f"param_model__{hyperparameter}"

    plot_df = optimization_cv_results[
        optimization_cv_results["model_name"] == model_name
    ].copy()

    if param_col not in plot_df.columns:
        raise ValueError(f"{param_col} was not found in optimization_cv_results.")

    # Average across secondary hyperparameters to isolate the effect of the selected one.
    plot_df = (
        plot_df
        .groupby(param_col, as_index=False)
```

```

        .agg(
            mean_train_rmse=("mean_train_rmse", "mean"),
            mean_cv_rmse=("mean_cv_rmse", "mean"),
            std_cv_rmse=("mean_cv_rmse", "std")
        )
        .sort_values(param_col)
    )

    best_value = optimization_results[model_name]["best_params"][f"model_{hyperparameter}"]

    plt.figure(figsize=(8, 5))

    plt.plot(
        plot_df[param_col],
        plot_df["mean_train_rmse"],
        marker="o",
        label="Training RMSE"
    )

    plt.plot(
        plot_df[param_col],
        plot_df["mean_cv_rmse"],
        marker="o",
        label="Validation RMSE"
    )

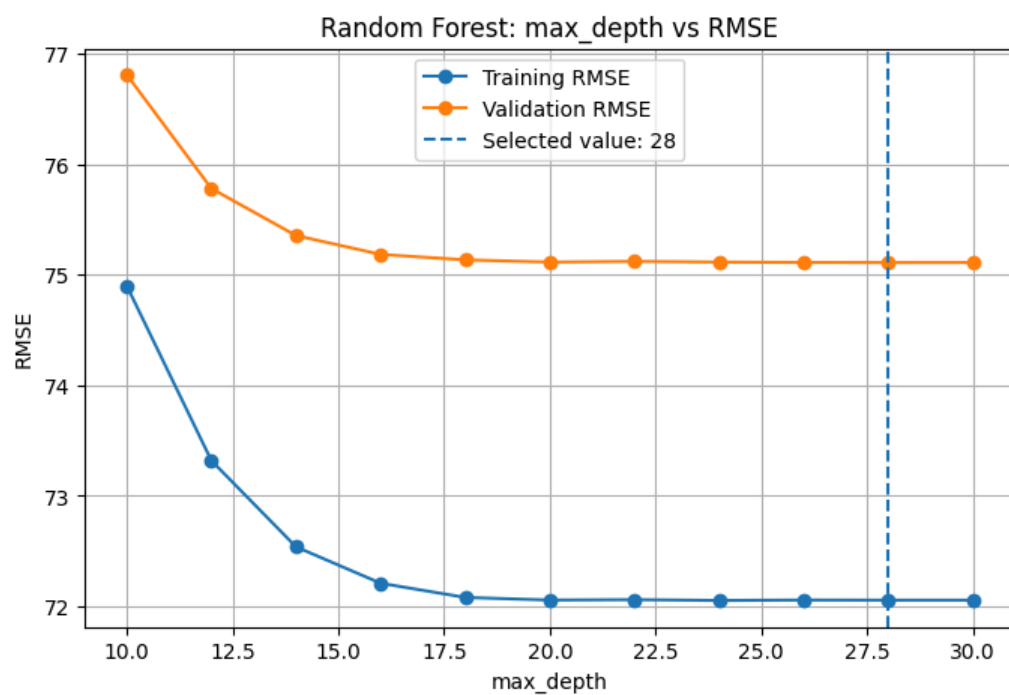
    plt.axvline(
        best_value,
        linestyle="--",
        label=f"Selected value: {best_value}"
    )

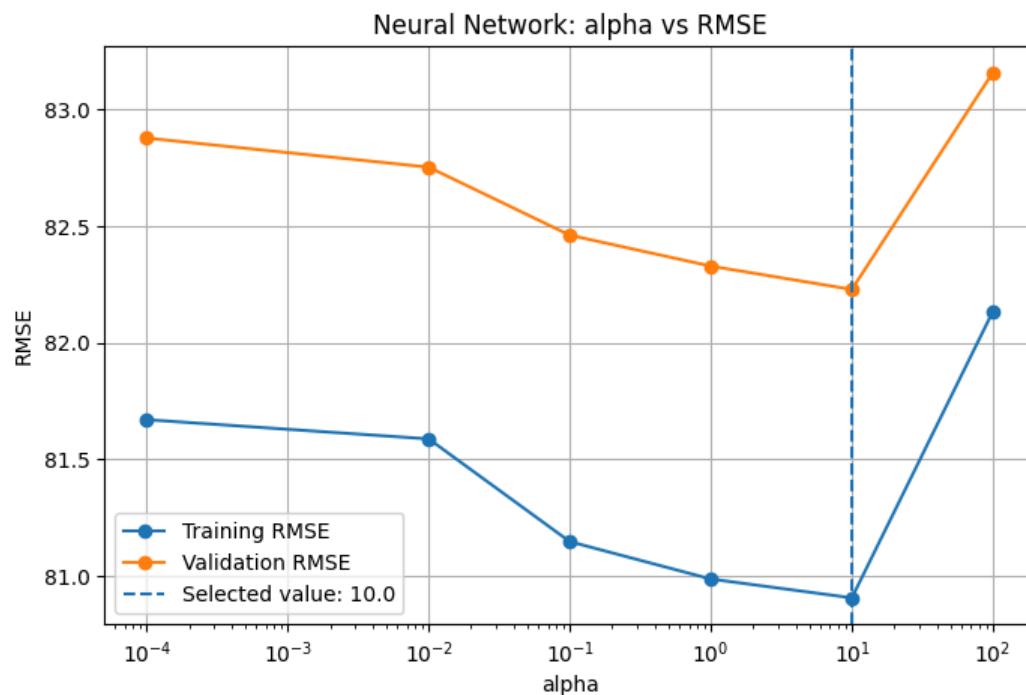
    if log_scale:
        plt.xscale("log")

    plt.title(f"{model_name}: {hyperparameter} vs RMSE")
    plt.xlabel(hyperparameter)
    plt.ylabel("RMSE")
    plt.legend()
    plt.grid(True)
    plt.show()

plot_hyperparameter_vs_loss("Random Forest", "max_depth")
plot_hyperparameter_vs_loss("Neural Network", "alpha", log_scale=True)

```





The optimization results indicate that the Random Forest clearly outperformed the Neural Network, achieving a lower validation RMSE (75.11 vs. 82.23) while also requiring substantially less execution time. The Random Forest curve shows that validation error decreases as `max_depth` increases and then stabilizes, suggesting that deeper trees improve performance up to a point without strong additional gains afterward. In contrast, the Neural Network benefits from stronger regularization, with the best result at `alpha = 10`, although its validation RMSE remains higher than Random Forest. Overall, these results support selecting Random Forest as the preferred model for the final evaluation.

```
In [ ]: import time
import numpy as np
import matplotlib.pyplot as plt

from sklearn.base import clone
from sklearn.model_selection import learning_curve, KFold
from sklearn.metrics import mean_squared_error, make_scorer

RANDOM_STATE = 42
LEARNING_CURVE_SAMPLE_SIZE = 240_000
CV_FOLDS_LEARNING_CURVE = 3

X_train = train_selected.drop(columns=[target])
y_train = train_selected[target]

# Use a representative subsample to keep the learning curve computationally feasible.
if len(X_train) > LEARNING_CURVE_SAMPLE_SIZE:
    lc_idx = X_train.sample(
        n=LEARNING_CURVE_SAMPLE_SIZE,
        random_state=RANDOM_STATE
    ).index
    X_lc = X_train.loc[lc_idx]
    y_lc = y_train.loc[lc_idx]
else:
    X_lc = X_train
    y_lc = y_train

print(f"Learning curve sample size: {X_lc.shape[0]:,} rows")
print(f"Number of features: {X_lc.shape[1]}")
print(f"CV folds: {CV_FOLDS_LEARNING_CURVE}")
print("Starting learning curve computation...\n")

rmse_scorer = make_scorer(
    lambda y_true, y_pred: np.sqrt(mean_squared_error(y_true, y_pred)),
    greater_is_better=False
)

cv_learning_curve = KFold(
    n_splits=CV_FOLDS_LEARNING_CURVE,
    shuffle=True,
    random_state=RANDOM_STATE
)
```

```

models_for_learning_curve = {
    "Random Forest": best_rf_model,
    "Neural Network": best_nn_model
}

learning_curve_results = {}

for model_name, model in models_for_learning_curve.items():
    print("=" * 70)
    print(f"Computing learning curve for: {model_name}")
    start_time = time.time()

    train_sizes, train_scores, cv_scores = learning_curve(
        estimator=clone(model),
        X=X_lc,
        y=y_lc,
        train_sizes=np.linspace(0.2, 1.0, 5),
        cv=cv_learning_curve,
        scoring=rmse_scorer,
        n_jobs=-1,
        shuffle=True,
        random_state=RANDOM_STATE
    )

    train_rmse = -train_scores
    cv_rmse = -cv_scores

    learning_curve_results[model_name] = {
        "train_sizes": train_sizes,
        "train_rmse_mean": train_rmse.mean(axis=1),
        "train_rmse_std": train_rmse.std(axis=1),
        "cv_rmse_mean": cv_rmse.mean(axis=1),
        "cv_rmse_std": cv_rmse.std(axis=1)
    }

    elapsed = time.time() - start_time
    print(f"Finished {model_name}")
    print(f"Elapsed time: {elapsed / 60:.2f} minutes\n")

```

Learning curve sample size: 240,000 rows

Number of features: 15

CV folds: 3

Starting learning curve computation...

```

=====
Computing learning curve for: Random Forest
Finished Random Forest
Elapsed time: 0.61 minutes

```

```

=====
Computing learning curve for: Neural Network
Finished Neural Network
Elapsed time: 3.03 minutes

```

```

In [ ]: def plot_learning_curve(model_name):
    results = learning_curve_results[model_name]

    train_sizes = results["train_sizes"]
    train_mean = results["train_rmse_mean"]
    train_std = results["train_rmse_std"]
    cv_mean = results["cv_rmse_mean"]
    cv_std = results["cv_rmse_std"]

    plt.figure(figsize=(8, 5))

    plt.plot(
        train_sizes,
        train_mean,
        marker="o",
        label="Training RMSE"
    )
    plt.fill_between(
        train_sizes,
        train_mean - train_std,
        train_mean + train_std,
        alpha=0.2
    )

```

```

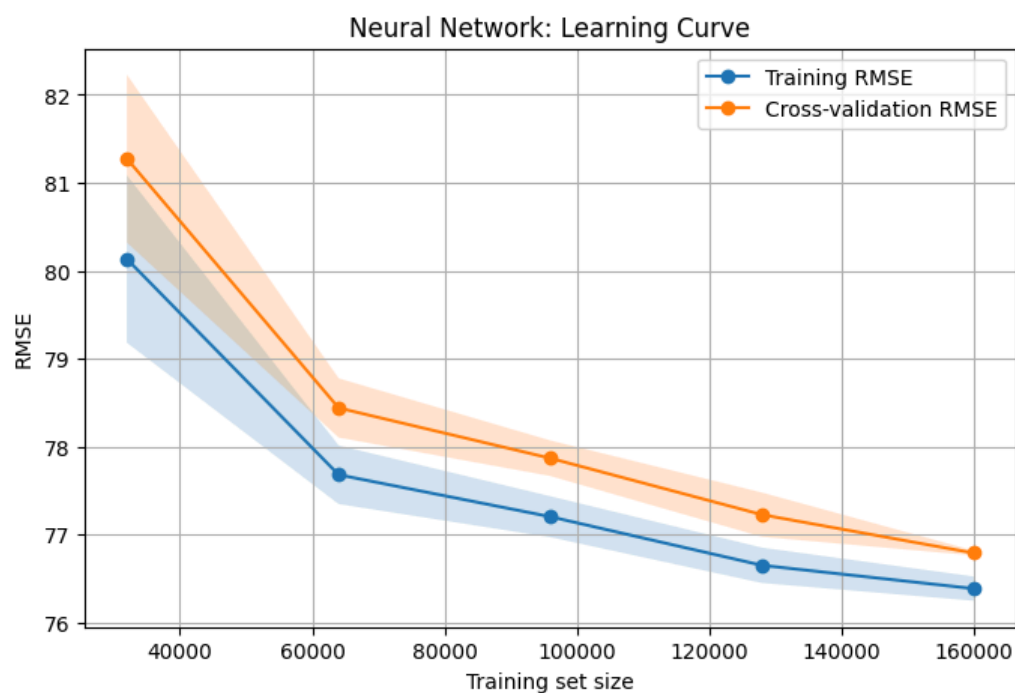
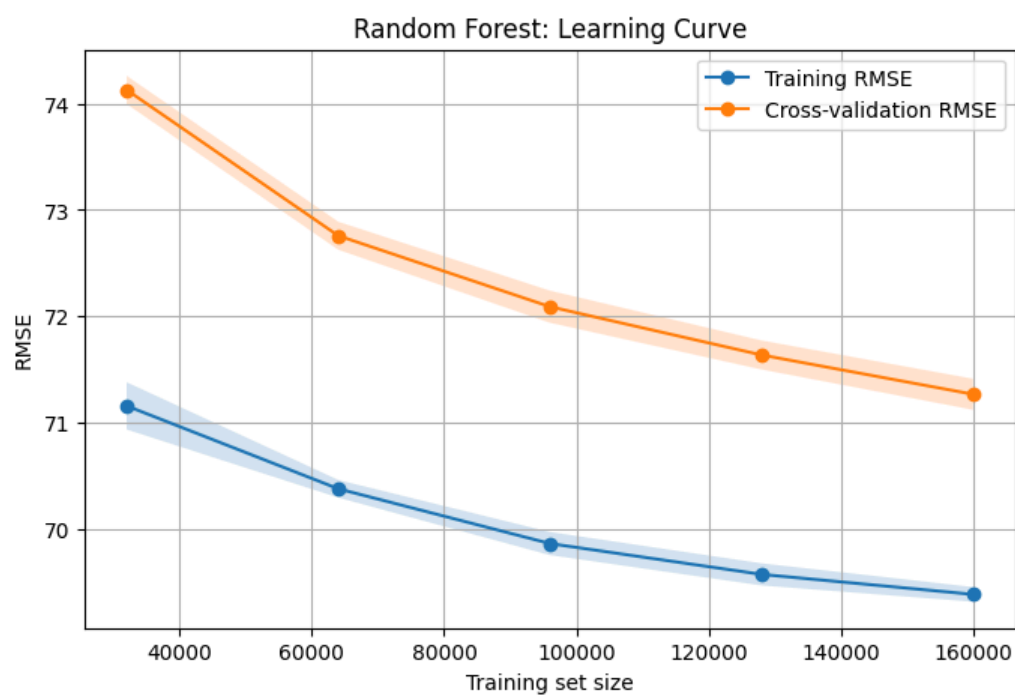
plt.plot(
    train_sizes,
    cv_mean,
    marker="o",
    label="Cross-validation RMSE"
)

plt.fill_between(
    train_sizes,
    cv_mean - cv_std,
    cv_mean + cv_std,
    alpha=0.2
)

plt.title(f"{model_name}: Learning Curve")
plt.xlabel("Training set size")
plt.ylabel("RMSE")
plt.legend()
plt.grid(True)
plt.show()

plot_learning_curve("Random Forest")
plot_learning_curve("Neural Network")

```



The learning curves show that both models improve as the training set size increases, since training and cross-validation RMSE decrease consistently. For Random Forest, the validation RMSE remains lower than the Neural Network across all training sizes, confirming its stronger predictive performance. However, the gap between training and validation RMSE suggests some remaining overfitting, although the gap becomes relatively stable with more data. For the Neural Network, the training and validation curves are closer together, indicating better bias-variance balance, but its overall error remains higher. Overall, additional training data could still improve both models, but Random Forest remains the preferred model based on lower cross-validation loss.

9. Statistical Comparison

The optimized Random Forest and Neural Network models are compared using repeated k-fold cross-validation. Because both models are evaluated on the same folds, their RMSE scores are paired; therefore, the Wilcoxon signed-rank test is used to assess whether the observed difference in predictive performance is statistically significant.

```
In [ ]: import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import joblib

from sklearn.base import clone
from sklearn.model_selection import RepeatedKFold
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from scipy.stats import wilcoxon

RANDOM_STATE = 42
STATISTICAL_TEST_SAMPLE_SIZE = 30_000
N_SPLITS = 5
N_REPEATS = 3

TARGET = 'PV_MATH_MEAN'

train_selected = pd.read_parquet("data/train_selected_features.parquet")

X_train = train_selected.drop(columns=[TARGET])
y_train = train_selected[TARGET]

# Use a representative subsample to make repeated cross-validation feasible.
if len(X_train) > STATISTICAL_TEST_SAMPLE_SIZE:
    stat_idx = X_train.sample(
        n=STATISTICAL_TEST_SAMPLE_SIZE,
        random_state=RANDOM_STATE
    ).index
    X_stat = X_train.loc[stat_idx]
    y_stat = y_train.loc[stat_idx]
else:
    X_stat = X_train
    y_stat = y_train

print(f"Statistical comparison sample size: {X_stat.shape[0]:,} rows")
print(f"Number of features: {X_stat.shape[1]}")
print(f"Repeated K-Fold: {N_SPLITS} folds x {N_REPEATS} repeats")
print(f"Total paired evaluations per model: {N_SPLITS * N_REPEATS}")
print("Starting repeated cross-validation...\n")

rkf = RepeatedKFold(
    n_splits=N_SPLITS,
    n_repeats=N_REPEATS,
    random_state=RANDOM_STATE
)

# import best models
best_rf_model = joblib.load("models/best_rf_model.pkl")
best_nn_model = joblib.load("models/best_nn_model.pkl")

models_to_compare = {
    "Random Forest": best_rf_model,
    "Neural Network": best_nn_model
}

statistical_results = []

start_total = time.time()

for fold_id, (train_idx, valid_idx) in enumerate(rkf.split(X_stat), start=1):
```

```

print("=" * 70)
print(f"Fold {fold_id}/{N_SPLITS * N_REPEATS}")

X_fold_train = X_stat.iloc[train_idx]
X_fold_valid = X_stat.iloc[valid_idx]
y_fold_train = y_stat.iloc[train_idx]
y_fold_valid = y_stat.iloc[valid_idx]

for model_name, model in models_to_compare.items():
    print(f"Training and evaluating: {model_name}")
    start_model = time.time()

    fitted_model = clone(model)
    fitted_model.fit(X_fold_train, y_fold_train)

    y_pred = fitted_model.predict(X_fold_valid)

    rmse = np.sqrt(mean_squared_error(y_fold_valid, y_pred))
    mae = mean_absolute_error(y_fold_valid, y_pred)
    r2 = r2_score(y_fold_valid, y_pred)

    elapsed_model = time.time() - start_model

    print(
        f"{model_name} | RMSE: {rmse:.4f} | "
        f"MAE: {mae:.4f} | R2: {r2:.4f} | "
        f"Time: {elapsed_model / 60:.2f} min"
    )

    statistical_results.append({
        "fold": fold_id,
        "model": model_name,
        "rmse": rmse,
        "mae": mae,
        "r2": r2,
        "elapsed_minutes": elapsed_model / 60
    })

elapsed_total = time.time() - start_total

statistical_results = pd.DataFrame(statistical_results)

print("=" * 70)
print("Repeated cross-validation completed.")
print(f"Total elapsed time: {elapsed_total / 60:.2f} minutes")

display(statistical_results.head())

```

Statistical comparison sample size: 30,000 rows
 Number of features: 15
 Repeated K-Fold: 5 folds x 3 repeats
 Total paired evaluations per model: 15
 Starting repeated cross-validation...

```

=====
Fold 1/15
Training and evaluating: Random Forest
Random Forest | RMSE: 75.8613 | MAE: 60.8914 | R2: 0.4076 | Time: 0.01 min
Training and evaluating: Neural Network

```

```

c:\Users\nicoc\AppData\Local\Programs\Python\Python310\lib\site-packages\sklearn\normal_network\_multilayer_perceptron.py:781: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (300) reached and the optimization has n't converged yet.
warnings.warn(

```

```

Neural Network | RMSE: 82.5013 | MAE: 66.3066 | R2: 0.2994 | Time: 0.33 min
=====

```

```

Fold 2/15
Training and evaluating: Random Forest
Random Forest | RMSE: 74.5333 | MAE: 59.5142 | R2: 0.4236 | Time: 0.01 min
Training and evaluating: Neural Network

```

```

c:\Users\nicoc\AppData\Local\Programs\Python\Python310\lib\site-packages\sklearn\normal_network\_multilayer_perceptron.py:781: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (300) reached and the optimization has n't converged yet.
warnings.warn(

```

Neural Network | RMSE: 81.9900 | MAE: 65.7331 | R2: 0.3024 | Time: 0.33 min

=====

Fold 3/15

Training and evaluating: Random Forest

Random Forest | RMSE: 75.6806 | MAE: 60.4077 | R2: 0.4209 | Time: 0.01 min

Training and evaluating: Neural Network

Neural Network | RMSE: 82.6192 | MAE: 65.7931 | R2: 0.3099 | Time: 0.33 min

=====

Fold 4/15

Training and evaluating: Random Forest

Random Forest | RMSE: 75.6375 | MAE: 60.7408 | R2: 0.4027 | Time: 0.01 min

Training and evaluating: Neural Network

c:\Users\nicoc\AppData\Local\Programs\Python\Python310\lib\site-packages\sklearn\neural_network_multilayer_perceptron.py:781: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (300) reached and the optimization has n't converged yet.

warnings.warn(

Neural Network | RMSE: 81.7632 | MAE: 65.6349 | R2: 0.3020 | Time: 0.32 min

=====

Fold 5/15

Training and evaluating: Random Forest

Random Forest | RMSE: 73.8361 | MAE: 59.2361 | R2: 0.4286 | Time: 0.01 min

Training and evaluating: Neural Network

Neural Network | RMSE: 82.2662 | MAE: 66.2764 | R2: 0.2907 | Time: 0.25 min

=====

Fold 6/15

Training and evaluating: Random Forest

Random Forest | RMSE: 75.0801 | MAE: 60.3118 | R2: 0.4107 | Time: 0.01 min

Training and evaluating: Neural Network

c:\Users\nicoc\AppData\Local\Programs\Python\Python310\lib\site-packages\sklearn\neural_network_multilayer_perceptron.py:781: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (300) reached and the optimization has n't converged yet.

warnings.warn(

Neural Network | RMSE: 81.2418 | MAE: 65.2417 | R2: 0.3100 | Time: 0.33 min

=====

Fold 7/15

Training and evaluating: Random Forest

Random Forest | RMSE: 74.5855 | MAE: 59.8447 | R2: 0.4277 | Time: 0.01 min

Training and evaluating: Neural Network

c:\Users\nicoc\AppData\Local\Programs\Python\Python310\lib\site-packages\sklearn\neural_network_multilayer_perceptron.py:781: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (300) reached and the optimization has n't converged yet.

warnings.warn(

Neural Network | RMSE: 81.8762 | MAE: 65.4434 | R2: 0.3104 | Time: 0.33 min

=====

Fold 8/15

Training and evaluating: Random Forest

Random Forest | RMSE: 75.5545 | MAE: 60.5031 | R2: 0.4109 | Time: 0.01 min

Training and evaluating: Neural Network

c:\Users\nicoc\AppData\Local\Programs\Python\Python310\lib\site-packages\sklearn\neural_network_multilayer_perceptron.py:781: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (300) reached and the optimization has n't converged yet.

warnings.warn(

Neural Network | RMSE: 82.6256 | MAE: 66.3880 | R2: 0.2955 | Time: 0.33 min

=====

Fold 9/15

Training and evaluating: Random Forest

Random Forest | RMSE: 74.1786 | MAE: 59.3662 | R2: 0.4201 | Time: 0.01 min

Training and evaluating: Neural Network

Neural Network | RMSE: 80.7753 | MAE: 64.8275 | R2: 0.3124 | Time: 0.33 min

=====

Fold 10/15

Training and evaluating: Random Forest

Random Forest | RMSE: 75.9694 | MAE: 60.7063 | R2: 0.4164 | Time: 0.01 min

Training and evaluating: Neural Network

c:\Users\nicoc\AppData\Local\Programs\Python\Python310\lib\site-packages\sklearn\neural_network_multilayer_perceptron.py:781: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (300) reached and the optimization has n't converged yet.

warnings.warn(

```

Neural Network | RMSE: 82.3704 | MAE: 65.8542 | R2: 0.3139 | Time: 0.33 min
=====
Fold 11/15
Training and evaluating: Random Forest
Random Forest | RMSE: 74.4575 | MAE: 60.0680 | R2: 0.4233 | Time: 0.01 min
Training and evaluating: Neural Network
Neural Network | RMSE: 81.0923 | MAE: 65.3640 | R2: 0.3160 | Time: 0.31 min
=====
Fold 12/15
Training and evaluating: Random Forest
Random Forest | RMSE: 75.4935 | MAE: 60.0792 | R2: 0.4269 | Time: 0.01 min
Training and evaluating: Neural Network
c:\Users\nicoc\AppData\Local\Programs\Python\Python310\lib\site-packages\sklearn\normalization\_multilayer_perceptron.py:781: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (300) reached and the optimization has n't converged yet.
  warnings.warn(
Neural Network | RMSE: 83.3733 | MAE: 66.5523 | R2: 0.3010 | Time: 0.32 min
=====
Fold 13/15
Training and evaluating: Random Forest
Random Forest | RMSE: 74.3781 | MAE: 59.4896 | R2: 0.4200 | Time: 0.01 min
Training and evaluating: Neural Network
Neural Network | RMSE: 83.1561 | MAE: 66.2868 | R2: 0.2750 | Time: 0.24 min
=====
Fold 14/15
Training and evaluating: Random Forest
Random Forest | RMSE: 76.0797 | MAE: 61.1884 | R2: 0.4044 | Time: 0.01 min
Training and evaluating: Neural Network
c:\Users\nicoc\AppData\Local\Programs\Python\Python310\lib\site-packages\sklearn\normalization\_multilayer_perceptron.py:781: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (300) reached and the optimization has n't converged yet.
  warnings.warn(
Neural Network | RMSE: 81.7670 | MAE: 65.9895 | R2: 0.3120 | Time: 0.32 min
=====
Fold 15/15
Training and evaluating: Random Forest
Random Forest | RMSE: 75.5098 | MAE: 60.3944 | R2: 0.4028 | Time: 0.01 min
Training and evaluating: Neural Network
Neural Network | RMSE: 81.7171 | MAE: 65.3879 | R2: 0.3006 | Time: 0.31 min
=====
Repeated cross-validation completed.
Total elapsed time: 4.80 minutes

```

	fold	model	rmse	mae	r2	elapsed_minutes
0	1	Random Forest	75.861322	60.891444	0.407609	0.006981
1	1	Neural Network	82.501251	66.306573	0.299369	0.325817
2	2	Random Forest	74.533305	59.514203	0.423555	0.006180
3	2	Neural Network	81.990046	65.733104	0.302443	0.325744
4	3	Random Forest	75.680604	60.407651	0.420948	0.006424

```

In [2]: # =====
# Wilcoxon signed-rank test
# =====
# Null hypothesis: both models have the same median RMSE across paired folds.
# Alternative hypothesis: the median RMSE differs between the two models.

comparison_summary = (
    statistical_results
    .groupby("model")
    .agg(
        mean_rmse=("rmse", "mean"),
        std_rmse=("rmse", "std"),
        mean_mae=("mae", "mean"),
        std_mae=("mae", "std"),
        mean_r2=("r2", "mean"),
        std_r2=("r2", "std"),
        mean_elapsed_minutes=("elapsed_minutes", "mean")
    )
    .sort_values("mean_rmse")
)

display(comparison_summary)

paired_rmse = statistical_results.pivot(
    index="fold",

```

```

        columns="model",
        values="rmse"
    )

    rf_rmse = paired_rmse["Random Forest"]
    nn_rmse = paired_rmse["Neural Network"]

    statistic, p_value = wilcoxon(rf_rmse, nn_rmse)

    alpha = 0.05

    mean_difference = rf_rmse.mean() - nn_rmse.mean()

    print("=" * 70)
    print("Wilcoxon signed-rank test")
    print("Metric: RMSE")
    print(f"Random Forest mean RMSE: {rf_rmse.mean():.4f}")
    print(f"Neural Network mean RMSE: {nn_rmse.mean():.4f}")
    print(f"Mean RMSE difference (RF - NN): {mean_difference:.4f}")
    print(f"Wilcoxon statistic: {statistic:.4f}")
    print(f"p-value: {p_value:.6f}")

    if p_value < alpha:
        print(
            f"Conclusion: The difference is statistically significant "
            f"at alpha = {alpha}."
        )
    else:
        print(
            f"Conclusion: There is not enough evidence to conclude that "
            f"the models differ significantly at alpha = {alpha}."
        )

    if mean_difference > 0:
        print("Direction: Neural Network achieved lower RMSE on average.")
    elif mean_difference < 0:
        print("Direction: Random Forest achieved lower RMSE on average.")
    else:
        print("Direction: Both models achieved the same average RMSE.")

```

	mean_rmse	std_rmse	mean_mae	std_mae	mean_r2	std_r2	mean_elapsed_minutes
model							
Random Forest	75.122353	0.727522	60.182792	0.594158	0.416444	0.009216	0.006271
Neural Network	82.075653	0.730715	65.805287	0.496196	0.303412	0.010757	0.313605

```

=====
Wilcoxon signed-rank test
Metric: RMSE
Random Forest mean RMSE: 75.1224
Neural Network mean RMSE: 82.0757
Mean RMSE difference (RF - NN): -6.9533
Wilcoxon statistic: 0.0000
p-value: 0.000061
Conclusion: The difference is statistically significant at alpha = 0.05.
Direction: Random Forest achieved lower RMSE on average.

```

```

In [ ]: plt.figure(figsize=(8, 5))

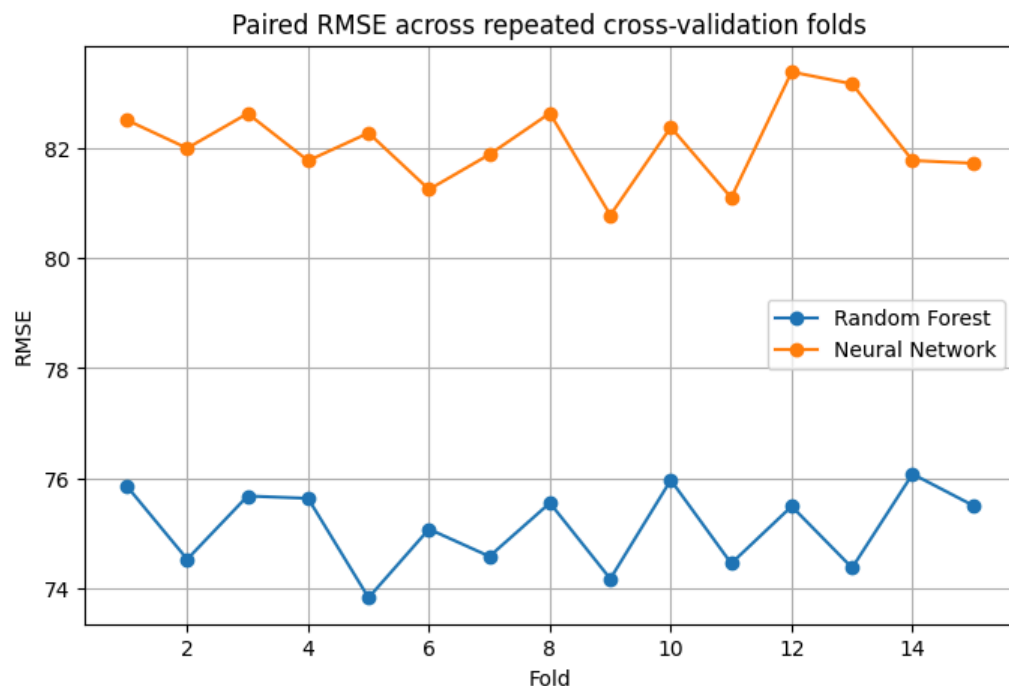
plt.plot(
    paired_rmse.index,
    paired_rmse["Random Forest"],
    marker="o",
    label="Random Forest"
)

plt.plot(
    paired_rmse.index,
    paired_rmse["Neural Network"],
    marker="o",
    label="Neural Network"
)

plt.title("Paired RMSE across repeated cross-validation folds")
plt.xlabel("Fold")
plt.ylabel("RMSE")
plt.legend()

```

```
plt.grid(True)
plt.show()
```



The Wilcoxon signed-rank test confirms that the performance difference between the two optimized models is statistically significant. Random Forest achieved a substantially lower mean RMSE than the Neural Network across the repeated cross-validation folds (75.12 vs. 82.08), with an average reduction of approximately 6.95 RMSE points. Since the p-value is well below 0.05, the null hypothesis of equal median performance is rejected. Therefore, the results provide strong evidence that Random Forest performs better than the Neural Network for this prediction task.

10. Final Test Evaluation

After feature selection, hyperparameter tuning, learning curve analysis, and statistical comparison, the selected final model is trained on the complete training set and evaluated once on the held-out test set. This provides the final estimate of generalization performance, since the test set has not been used during model selection.

```
In [ ]: import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import joblib
import os

from sklearn.base import clone
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

TARGET = "PV_MATH_MEAN"

train_selected = pd.read_parquet("data/train_selected_features.parquet")
test_selected = pd.read_parquet("data/test_selected_features.parquet")

X_train_final = train_selected.drop(columns=[TARGET])
y_train_final = train_selected[TARGET]
X_test_final = test_selected.drop(columns=[TARGET])
y_test_final = test_selected[TARGET]

# Select the final model using the lowest average RMSE from repeated cross-validation.
if "comparison_summary" in globals():
    final_model_name = comparison_summary.index[0]
elif "best_model_name" in globals():
    final_model_name = best_model_name
else:
    raise NameError(
        "Neither comparison_summary nor best_model_name is available. "
        "Run the model comparison or hyperparameter optimization section first."
    )

# Retrieve the selected model from memory when available; otherwise, load it from disk.
```

```

if final_model_name == "Random Forest":
    if "best_rf_model" in globals():
        selected_model = best_rf_model
    else:
        selected_model = joblib.load("models/best_rf_model.pkl")
elif final_model_name == "Neural Network":
    if "best_nn_model" in globals():
        selected_model = best_nn_model
    else:
        selected_model = joblib.load("models/best_nn_model.pkl")
else:
    raise ValueError(f"Unexpected final model name: {final_model_name}")

final_model = clone(selected_model)

print("=" * 70)
print("Final test evaluation")
print(f"Selected final model: {final_model_name}")
print(f"Training rows: {X_train_final.shape[0]:,}")
print(f"Test rows: {X_test_final.shape[0]:,}")
print(f"Number of features: {X_train_final.shape[1]}")
print("Training the final model on the complete training set...")

start_time = time.time()
final_model.fit(X_train_final, y_train_final)
y_test_pred = final_model.predict(X_test_final)
elapsed = time.time() - start_time

test_rmse = np.sqrt(mean_squared_error(y_test_final, y_test_pred))
test_mae = mean_absolute_error(y_test_final, y_test_pred)
test_r2 = r2_score(y_test_final, y_test_pred)

test_results = pd.DataFrame({
    "model": [final_model_name],
    "test_rmse": [test_rmse],
    "test_mae": [test_mae],
    "test_r2": [test_r2],
    "elapsed_minutes": [elapsed / 60]
})

os.makedirs("models", exist_ok=True)
joblib.dump(final_model, "models/final_test_model.pkl")

test_predictions = pd.DataFrame({
    "actual": y_test_final.to_numpy(),
    "predicted": y_test_pred,
    "residual": y_test_final.to_numpy() - y_test_pred
})

test_predictions.to_csv("models/final_test_predictions.csv", index=False)
test_results.to_csv("models/final_test_results.csv", index=False)

print("Final test evaluation completed.")
print(f"Elapsed time: {elapsed / 60:.2f} minutes")
print(f"Test RMSE: {test_rmse:.4f}")
print(f"Test MAE: {test_mae:.4f}")
print(f"Test R2: {test_r2:.4f}")
print("Saved outputs:")
print("- models/final_test_model.pkl")
print("- models/final_test_predictions.csv")
print("- models/final_test_results.csv")

display(test_results)

# Diagnostic plot: predicted vs. actual values
plt.figure(figsize=(6, 6))
plt.scatter(y_test_final, y_test_pred, alpha=0.25, s=10)

min_value = min(y_test_final.min(), y_test_pred.min())
max_value = max(y_test_final.max(), y_test_pred.max())
plt.plot(
    [min_value, max_value],
    [min_value, max_value],
    linestyle="--",
    label="Perfect prediction"
)

plt.title(f"{final_model_name}: Predicted vs Actual Values")
plt.xlabel("Actual PV_MATH_MEAN")
plt.ylabel("Predicted PV_MATH_MEAN")
plt.legend()

```

```

plt.grid(True)
plt.show()

# Diagnostic plot: residuals
residuals = y_test_final - y_test_pred

plt.figure(figsize=(8, 5))
plt.scatter(y_test_pred, residuals, alpha=0.25, s=10)
plt.axhline(0, linestyle="--")
plt.title(f"{final_model_name}: Residual Plot")
plt.xlabel("Predicted PV_MATH_MEAN")
plt.ylabel("Residuals")
plt.grid(True)
plt.show()

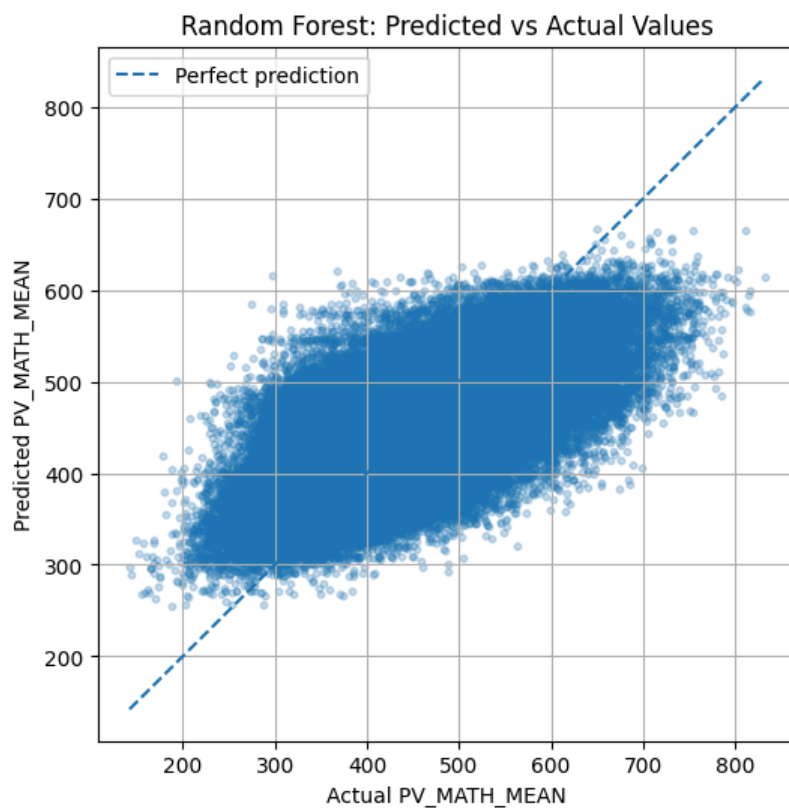
```

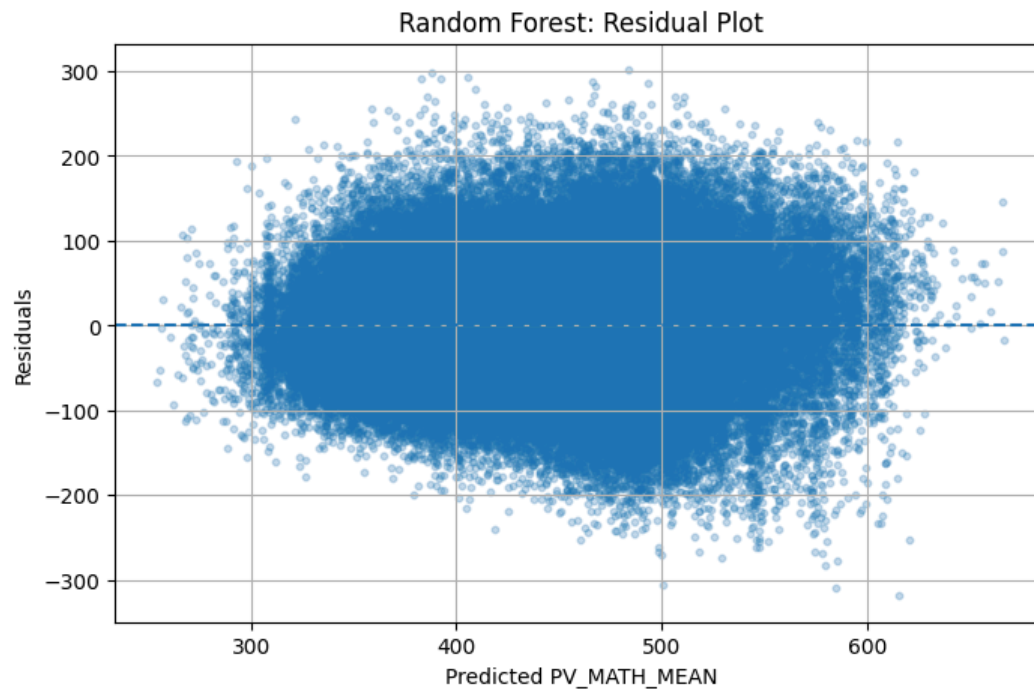
```

=====
Final test evaluation
Selected final model: Random Forest
Training rows: 490,995
Test rows: 122,749
Number of features: 15
Training the final model on the complete training set...
Final test evaluation completed.
Elapsed time: 0.28 minutes
Test RMSE: 69.8210
Test MAE: 55.3963
Test R2: 0.4975
Saved outputs:
- models/final_test_model.pkl
- models/final_test_predictions.csv
- models/final_test_results.csv

```

	model	test_rmse	test_mae	test_r2	elapsed_minutes
0	Random Forest	69.821026	55.396345	0.497469	0.284992





The final test evaluation confirms that the selected Random Forest model generalizes well to unseen data, achieving a test RMSE of 69.82, MAE of 55.40, and an R^2 of 0.4975. This indicates that the model explains approximately half of the variance in mathematics performance, which is reasonable given that only 15 selected features were used. The predicted vs. actual plot shows a clear positive relationship, although predictions are compressed toward the center of the score range, meaning the model tends to underpredict very high scores and overpredict very low scores. The residual plot is broadly centered around zero, but the spread of residuals suggests that there is still unexplained variability, likely due to factors not captured by the selected predictors. Overall, the final model provides a solid predictive baseline, but additional student-level, socioeconomic, and school-level features could further improve performance.

11. Conclusions

This project developed a complete machine learning workflow to predict students' mathematics performance (`PV_MATH_MEAN`) using a large-scale encoded dataset. After applying feature selection with mutual information, the final modeling stage focused on a compact set of 15 predictors that mainly captured geographic, institutional, linguistic, and demographic information. The exploratory analysis showed that the target variable had sufficient variability for regression modeling, while the weak individual correlations between predictors and the target suggested that mathematics performance is influenced by multiple interacting factors rather than by a single dominant variable.

The comparison between models showed that Random Forest consistently outperformed the Neural Network. During hyperparameter optimization, Random Forest achieved a lower cross-validation RMSE and required substantially less computational time. The learning curves also confirmed that Random Forest maintained better generalization performance across different training set sizes, although both models benefited from additional data. The Wilcoxon signed-rank test provided statistical evidence that the difference between the two models was significant, supporting the selection of Random Forest as the final model.

In the final test evaluation, the Random Forest model achieved an RMSE of 69.82, an MAE of 55.40, and an R^2 of 0.4975. These results indicate that the model explains approximately half of the variability in mathematics performance, which represents a solid predictive baseline given the limited number of selected features. However, the diagnostic plots showed that predictions tend to be compressed toward the average score range, suggesting that the model has more difficulty predicting extreme low and high performance values. Overall, the results demonstrate that the selected features contain meaningful predictive information, but future improvements could be achieved by incorporating richer student-level, socioeconomic, and school-level variables.

References

[1] Organisation for Economic Co-operation and Development (OECD), "Student Performance (PISA)," OECD. [Online]. Available: <https://www.oecd.org/en/topics/student-performance-pisa.html>. [Accessed: Jun. 21, 2026].